

Aula 13 - TLS

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
November 27, 2025

Protegendo conexões TCP: TLS

- **Contextualização:**
 - Vimos segurança na camada de rede.
 - Agora, subimos uma camada na pilha de protocolo.
- **O Objetivo do TLS:**
 - Examinar como a criptografia pode aprimorar o **TCP**.
 - (Transport Layer Security).
- **Serviços de Segurança Adicionados ao TCP:**
 - Sigilo (Confidencialidade).
 - Integridade dos Dados.
 - Autenticação do Ponto Final (End-point Authentication).
- **Padronização e Histórico:**
 - O TLS é padronizado pelo IETF na **RFC 4346**.
 - Uma versão anterior e semelhante deste protocolo é o **SSL (Secure Sockets Layer)** versão 3.

TLS: Histórico e Adoção

- **Origem:** O SSL (Secure Sockets Layer) foi originalmente projetado pela Netscape.
 - (Ideias básicas sobre proteção do TCP antecedem este trabalho, ex: Woo, 1994).
- **Sucessor:** O TLS (Transport Layer Security) é o sucessor padronizado do SSL.
- **Adoção Massiva:**
 - Suportado por todos os navegadores Web e servidores Web populares.
 - Usado pelo Gmail e por **basicamente todos os sites de comércio eletrônico** (Amazon, eBay, TaoBao).
 - Centenas de bilhões de dólares são gastos anualmente usando TLS.
- **Como o usuário identifica o TLS?**
 - Se você já comprou algo online com cartão de crédito, sua comunicação foi quase certamente via TLS.
 - A URL no navegador se inicia com **https://** (em vez de http://).

A Necessidade do TLS: Cenário de E-commerce

- **O Cenário Típico:**

- **Bob** (cliente) está navegando na Web.
- Ele acessa o site da **Alice Incorporated** (vendedor) para comprar perfume.

- **A Ação:**

- O site exibe um formulário.
- Bob insere informações sensíveis:
 - Tipo de perfume e quantidade.
 - Seu endereço.
 - O **número de seu cartão de crédito**.
- Bob clica em "Enviar".

- **A Expectativa (sem segurança):**

- Bob espera receber o perfume e a cobrança correta.
- **O Risco:** Se nenhuma medida de segurança for tomada, Bob pode ter "surpresas".

Riscos: O Cenário de E-commerce sem TLS

- **1. Se nenhuma Confidencialidade (Sigilo) for utilizada:**
 - Um invasor (Trudy) poderia **interceptar** o pedido.
 - Trudy rouba as informações do cartão de crédito de Bob.
 - Consequência: O invasor pode fazer compras à custa de Bob.
- **2. Se nenhuma Integridade dos Dados for utilizada:**
 - Um invasor poderia **modificar** o pedido de Bob em trânsito.
 - Ex: Fazer Bob comprar 10x mais frascos de perfume do que o desejado.
- **3. Se nenhuma Autenticação do Servidor for utilizada:**
 - Um servidor (mantido por Trudy) poderia **se passar** pela Alice Inc., usando o mesmo logotipo.
 - Bob insere seus dados no site falso de Trudy.
 - Consequência: Trudy fica com o dinheiro e some, ou pratica **roubo de identidade** (nome, endereço, cartão).

TLS: Além do HTTP e Posição na Pilha

- **Uso Comum: HTTPS**

- Muitas vezes, o TLS é usado para oferecer segurança em transações que ocorrem pelo HTTP.

- **O Verdadeiro Alcance: Qualquer Aplicação TCP**

- Como o TLS protege o **TCP**, ele pode ser empregado por **qualquer aplicação** que execute sobre o TCP (ex: FTP, SMTP, etc.).

- **Interface para o Desenvolvedor (API)**

- O TLS provê uma Interface de Programação de Aplicações (API) com Sockets, muito semelhante à API de Sockets do TCP.
- Para usar o TLS, a aplicação simplesmente inclui as classes ou bibliotecas SSL/TLS.

- **Localização na Pilha de Protocolos**

- Tecnicamente, o TLS reside na **Camada de Aplicação**.
- Do ponto de vista do **desenvolvedor**, ele é visto como um **protocolo de transporte** que provê serviços do TCP aprimorados com serviços de segurança.
- (Conforme Figura 8.24: TLS fica entre a Aplicação e o TCP).

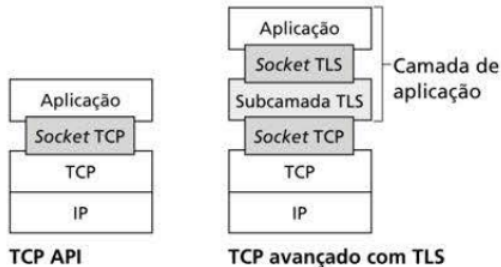


Figura 8.24 Embora o TLS resida tecnicamente na camada de aplicação, do ponto de vista do desenvolvedor, ele é um protocolo da camada transporte.

Arquitetura do TLS

- O TLS é projetado para **usar o TCP** para prover um serviço seguro, confiável (reliable) e fim-a-fim.
- O TLS não é um protocolo único, mas sim **duas camadas** de protocolos.
- **Camada 1: O Protocolo de Registro (Record Protocol)**
 - Fica "em cima" do TCP.
 - Fornece os serviços básicos de segurança (criptografia, integridade) para os protocolos da camada superior.
- **Camada 2: Protocolos Superiores (Gerenciamento TLS)**
 - São protocolos **específicos do TLS**, usados na **gestão** das trocas TLS.
 - São três protocolos que rodam "em cima" do Record Protocol:
 - Protocolo Handshake (Handshake Protocol)
 - Protocolo de Troca de Especificação de Cifra (Change Cipher Spec Protocol)
 - Protocolo de Alerta (Alert Protocol)
- **Camada de Aplicação (Exemplo)**
 - O **HTTP** (Hypertext Transfer Protocol), usado para interação Web cliente/servidor, pode operar "em cima" do TLS.

A pilha SSL/TLS

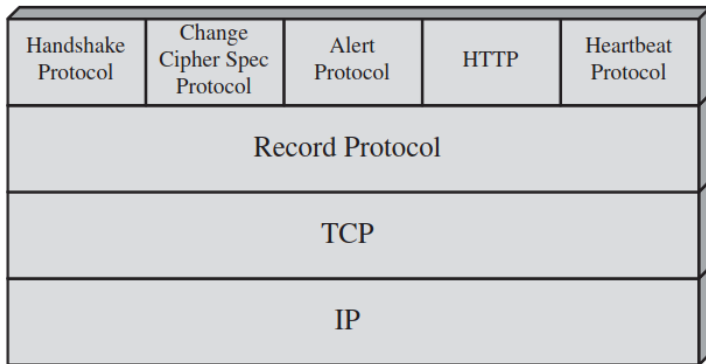


Figure 22.4 SSL/TLS Protocol Stack

Conceitos Importantes: Conexão TLS vs. Sessão TLS

- Existem dois conceitos centrais: Conexão e Sessão.
- **Conexão (Connection)**
 - É um transporte (no sentido OSI) que provê um serviço.
 - No TLS, é uma relação *peer-to-peer*.
 - As conexões são **transitórias** (passageiras).
 - **Toda conexão está associada a UMA sessão.**
- **Sessão (Session)**
 - É uma associação entre um cliente e um servidor.
 - Criada pelo **Handshake Protocol**.
 - Define um conjunto de **parâmetros de segurança criptográficos** (chaves, algoritmos, etc.).
 - Estes parâmetros podem ser **compartilhados** entre múltiplas conexões.
 - **Objetivo:** Evitar a negociação (cara) de novos parâmetros de segurança para cada nova conexão.
- **Relacionamento:**
 - Pode haver **múltiplas conexões seguras** entre um cliente e um servidor.
 - Na prática, essas múltiplas conexões compartilham (reutilizam) os parâmetros de uma **única sessão**.

O protocolo de registro do TLS

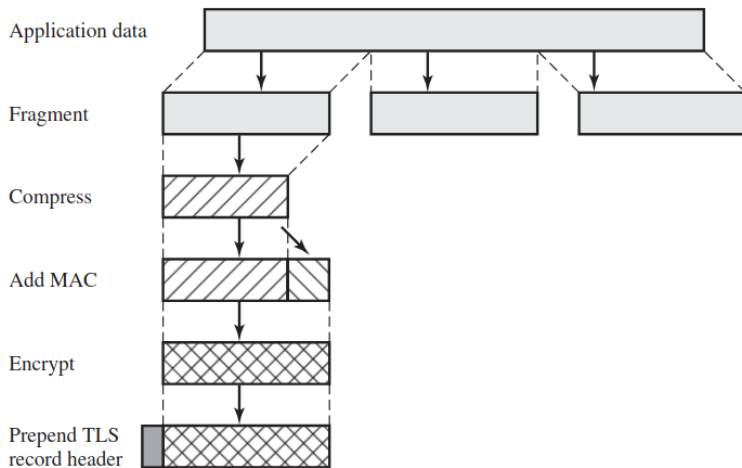


Figure 22.5 TLS Record Protocol Operation

Protocolo de Registro TLS (Record Protocol): Serviços

- O Protocolo de Registro (SSL/TLS Record Protocol) provê dois serviços básicos para as conexões TLS:
- **1. Confidencialidade (Sigilo)**
 - O Protocolo Handshake (visto mais tarde) define uma **chave secreta compartilhada**.
 - Esta chave é usada para **criptografia simétrica** dos payloads (dados) do TLS.
- **2. Integridade da Mensagem**
 - O Protocolo Handshake também define (outra) **chave secreta compartilhada**.
 - Esta chave é usada para formar um **MAC (Message Authentication Code)**.
- **Tipos de Conteúdo (Content Types) transportados:**
 - *change_cipher_spec* (Protocolo de Gerenciamento TLS)
 - *alert* (Protocolo de Gerenciamento TLS)
 - *handshake* (Protocolo de Gerenciamento TLS)
 - *application_data* (Dados da Aplicação, ex: HTTP).
- **Obs:** O conteúdo de *application_data* é **opaco** para o TLS (ele não sabe o que é HTTP, apenas o protege).

Protocolo Change Cipher Spec

- Esse protocolo utiliza o Protocolo de Registro (TLS Record Protocol).
- É o **mais simples** de todos os protocolos de gerenciamento do TLS.
- **Estrutura do Protocolo:**
 - Consiste em uma **única mensagem**.
 - Essa mensagem contém apenas **um byte**.
 - O valor desse byte é **1**.
- **Finalidade (Ação Chave):**
 - A sua única finalidade é sinalizar uma transição.
 - A mensagem "Change Cipher Spec" **NÃO** é parte do Handshake, mas sim um sinal *entre* fases.
- **O Que a Mensagem Causa?**
 - Atualiza (ativa) o cipher suite a ser usado nesta conexão a partir deste ponto.

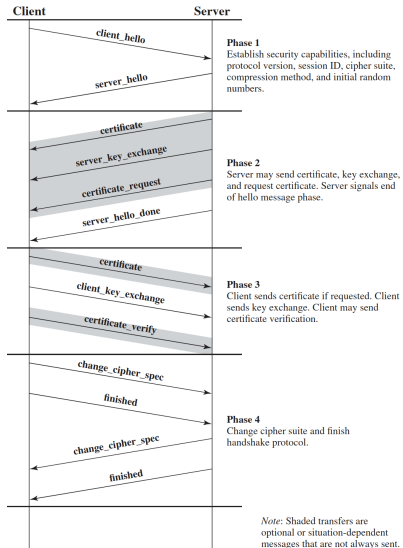
Protocolo de Alerta (Alert Protocol)

- **Finalidade:** Usado para transmitir alertas relacionados ao TLS para o outro lado.
- **Segurança:** Assim como os dados da aplicação, as mensagens de alerta são **comprimadas e criptografadas**.
- **Estrutura da Mensagem (2 bytes):**
 - **Primeiro Byte (Severidade):**
 - 'warning(1)': Um aviso (não fatal).
 - 'fatal(2)': Um erro fatal.
 - **Segundo Byte (Código):**
 - Indica o alerta específico (ex: 'bad_record_mac', 'close_notify').
- **Consequência de Alertas Fatais:**
 - Se o nível for 'fatal', o TLS **termina a conexão imediatamente**.
 - Outras conexões na mesma sessão podem continuar.
 - Nenhuma *nova conexão* pode ser estabelecida nesta sessão.
- **Exemplos de Alertas:**
 - **Exemplo Fatal:** 'incorrect MAC' (MAC incorreto).
 - **Exemplo Não Fatal (Warning):** 'close_notify' (notifica o destinatário que o remetente não enviará mais mensagens nesta conexão).

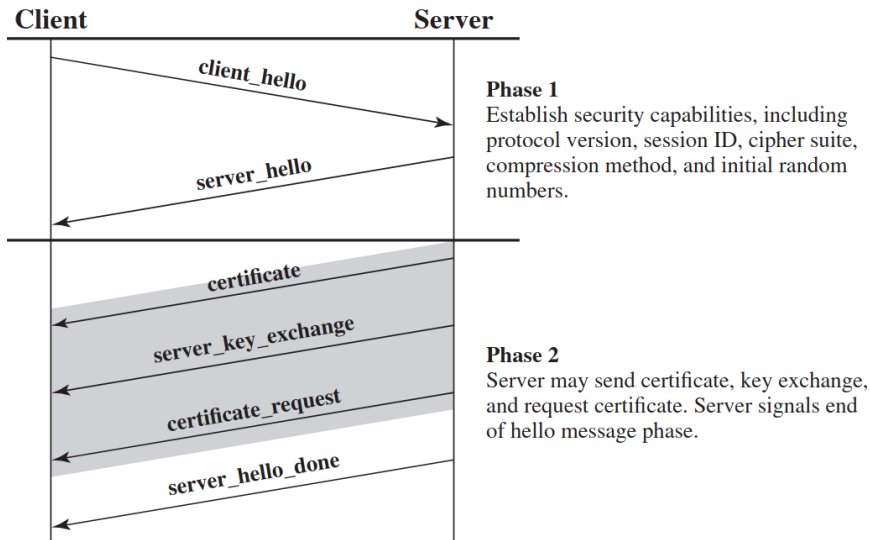
Protocolo Handshake (Handshake Protocol)

- É a parte **mais complexa** do TLS.
- Usado **antes** que quaisquer dados da aplicação sejam transmitidos.
- **Permite que o Servidor e o Cliente:**
 - Autentiquem-se mutuamente (especialmente o servidor ao cliente).
 - Negociem um algoritmo de **criptografia**.
 - Negociem um algoritmo de **MAC** (integridade).
 - Negociem as **chaves criptográficas** a serem usadas para proteger os dados.
- Consiste em uma série de mensagens trocadas entre cliente e servidor, divididas em quatro fases (conforme Figura 22.6).

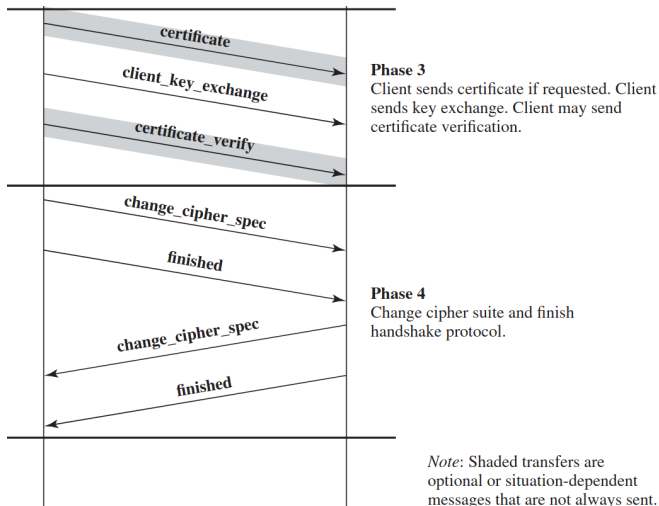
Protocolo Handshake do TLS



Protocolo Handshake do TLS (zoom 1)



Protocolo Handshake do TLS (zoom 2)



2.6 Handshake Protocol Action

As Quatro Fases do Handshake (Visão Geral)

- O protocolo consiste em uma série de mensagens trocadas.
- A troca pode ser vista como tendo quatro fases distintas:
- **Fase 1: Iniciação e Capacidades**
 - Inicia a conexão lógica (`client_hello`, `server_hello`).
 - Estabelece as capacidades de segurança (o que ambos suportam).
- **Fase 2: Autenticação do Servidor**
 - O servidor envia seu certificado e (possivelmente) informações de chave.
 - Termina com `server_done`.
- **Fase 3: Resposta do Cliente**
 - O cliente verifica o servidor e envia suas informações de chave (ex: `ClientKeyExchange`).
- **Fase 4: Finalização**
 - Completa a configuração da conexão segura, ativando os novos parâmetros (`ChangeCipherSpec`, `Finished`).

Handshake - Fase 1 (Início: Client Hello)

- A troca é iniciada pelo **cliente**, que envia uma mensagem `client_hello`.
- **Parâmetros do `client_hello`:**
- **Version:** A **versão mais alta** do TLS que o cliente entende.
- **Random:** Uma estrutura aleatória gerada pelo cliente.
 - Composição: 32-bit timestamp + 28 bytes aleatórios (de um gerador seguro).
 - Usado para prevenir ataques de *replay*.
- **Session ID:**
 - **Valor diferente de zero:** Cliente deseja atualizar parâmetros de uma conexão existente ou criar nova conexão nesta sessão (retomada de sessão).
 - **Valor zero:** Cliente deseja estabelecer uma nova conexão em uma nova sessão.
- **CipherSuite:**
 - Uma **lista** das combinações de algoritmos criptográficos que o cliente suporta.
 - Em **ordem decrescente de preferência**.
 - Cada item define: algoritmo de troca de chave E um CipherSpec (algoritmo de cifra/MAC).
- **Compression method:** Lista de métodos de compressão suportados.

Handshake - Fase 1 (Resposta: Server Hello)

- Após enviar o `client_hello`, o cliente aguarda a resposta.
- O servidor responde com uma mensagem `server_hello`.
- **Conteúdo do `server_hello`:**
 - Contém os **mesmos parâmetros** do `client_hello`.
 - **Porém:** O servidor agora **selecionou UMA opção** de cada lista (as que serão usadas).
 - **Version:** A versão que será usada (escolhida pelo servidor).
 - **Random:** O valor aleatório do **servidor**.
 - **Session ID:** O ID da sessão (se for retomada ou nova).
 - **CipherSuite:** A **única** suíte de cifras (escolhida pelo servidor) que será usada.
 - **Compression method:** O **único** método (escolhido pelo servidor) que será usado.

Handshake - Fase 2 (Autenticação do Servidor)

- Os detalhes desta fase dependem do esquema de criptografia de chave pública (definido no CipherSuite).
- **O que o servidor envia (após o `server_hello`):**
 - **1. Certificado (Certificate):**
 - (Quase sempre) O servidor envia seu certificado digital para o cliente se autenticar.
 - **2. Informação de Chave (ServerKeyExchange):**
 - (Opcional) Pode enviar informações adicionais de chave (ex: parâmetros Diffie-Hellman).
 - **3. Solicitação de Certificado (CertificateRequest):**
 - (Opcional) O servidor pode solicitar um certificado do cliente (para autenticação mútua).
 - **4. Fim da Mensagem (Server Done):**
 - **Sempre obrigatória.**
 - O servidor envia a mensagem `server_done`.
 - Indica o fim do "olá" do servidor e mensagens associadas.
 - Após enviar, o servidor aguarda a resposta do cliente.

Handshake - Fase 3 (Resposta do Cliente)

- Após receber o `server_done` do servidor:
- **1. Verificação (Ações do Cliente):**
 - O cliente deve verificar se o servidor forneceu um **certificado válido** (se foi exigido).
 - O cliente deve checar se os parâmetros do `server_hello` (CipherSuite, etc.) são aceitáveis.
- **2. Envio de Mensagens (Ações do Cliente):**
 - Se tudo estiver satisfatório, o cliente envia uma ou mais mensagens ao servidor.
 - (Ex: `ClientKeyExchange` - contendo o "pre-master secret" criptografado com a chave pública do servidor).
 - (Ex: `CertificateVerify` - se o servidor solicitou um certificado do cliente e o cliente está respondendo).

Handshake - Fase 4 (Finalização - Ativação)

- **1. Ação do Cliente:**

- Envia a mensagem `change_cipher_spec`.
- (Esta mensagem usa o *Change Cipher Spec Protocol*, não o Handshake Protocol).
- Ação: Copia o CipherSpec *pendente* (negociado) para o CipherSpec *atual* (em uso).
- Imediatamente, envia a mensagem `finished`.
- **Importante:** A mensagem `finished` já é enviada usando os **novos** algoritmos, chaves e segredos.
- A `finished` verifica se a troca de chaves e a autenticação foram bem-sucedidas.

- **2. Ação do Servidor (em resposta):**

- Envia sua própria mensagem `change_cipher_spec`.
- Transfere o CipherSpec *pendente* para o atual.
- Envia sua própria mensagem `finished` (também já criptografada).

- **3. Conclusão:**

- Neste ponto, o handshake está completo.
- Cliente e servidor podem começar a trocar dados da camada de aplicação (HTTP) de forma segura.

Protocolo Heartbeat: Definição e Contexto

- **O que é um "Heartbeat"?** (Contexto Geral)
 - Um sinal periódico (hardware ou software) usado para indicar operação normal ou para sincronizar partes de um sistema.
- **O que é um "Protocolo Heartbeat"?**
 - Usado para monitorar a **disponibilidade** de uma entidade de protocolo (verificar se o outro lado está "vivo").
- **No contexto SSL/TLS:**
 - Foi definido em 2012 na **RFC 6250**.
 - Nome oficial: "Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) Heartbeat Extension".

Protocolo Heartbeat: Propósitos

- O heartbeat serve a dois propósitos principais:
- **Propósito 1: "Keep-Alive" (Verificação de Atividade)**
 - Assegura ao remetente que o destinatário ainda está "vivo".
 - É útil mesmo que não tenha havido atividade (envio de dados da aplicação) sobre a conexão TCP subjacente por um tempo.
- **Propósito 2: "Firewall Traversal" (Evitar Ociosidade)**
 - Gera atividade (tráfego) na conexão durante períodos ociosos (idle).
 - **Objetivo:** Evitar o fechamento da conexão por *firewalls* que não toleram conexões ociosas.

Protocolo Heartbeat: Operação e Negociação

- **Posição na Pilha:**
 - Roda "em cima" (on top) do Protocolo de Registro TLS (TLS Record Protocol).
- **Tipos de Mensagem (2):**
 - 'heartbeat_request'
 - 'heartbeat_response'
- **Negociação (Como é ativado?):**
 - O uso do Heartbeat é estabelecido durante a **Fase 1 do Protocolo Handshake**.
 - Cada lado (peer) indica se suporta heartbeats.
 - Se suportado, o lado indica seu modo de operação:
Modo 1 Disposto a *receber requests* e *responder* com responses.
Modo 2 Disposto *apenas a enviar* requests.

Protocolo Heartbeat: Estrutura da Mensagem

- **Fluxo:**
 - Uma 'heartbeat_request' pode ser enviada a qualquer momento.
 - Ao receber um *request*, o receptor deve responder *prontamente* com um 'heartbeat_response'.
- **Conteúdo da Mensagem request:**
 - **Payload (Carga Útil):**
 - Um conteúdo **aleatório**.
 - Tamanho: entre 16 bytes e 64 Kbytes.
 - **Payload Length (Comprimento):** O tamanho do payload.
 - **Padding (Preenchimento):** Um conteúdo aleatório adicional.
- **Requisito da Mensagem response:**
 - Deve incluir uma **cópia exata** do *payload* recebido no *request*.
- **Uso do Padding:**
 - Permite a descoberta de Path MTU (PMTU).
 - (Enviando *requests* com *padding* crescente até que a resposta falhe).

Ataques SSL/TLS: Introdução e Evolução

- Desde a introdução do SSL (1994) e a padronização do TLS, inúmeros ataques foram criados.
- **Ciclo de Ameaça e Resposta:**
 - A aparição de cada ataque necessitou mudanças (contramedidas) no protocolo, nas ferramentas de criptografia ou nas implementações.
- **Categorias Gerais de Ataque:**
 - 1. Ataques ao Protocolo Handshake.
 - 2. Ataques aos protocolos de Registro (Record) e Dados da Aplicação.
 - 3. Ataques à PKI (Infraestrutura de Chave Pública).
 - 4. Outros ataques (ex: DoS).
- **A Realidade dos Protocolos de Internet:**
 - Um protocolo "perfeito" ou implementação "perfeita" nunca é alcançado.
 - A evolução dos protocolos é determinada por um constante "vai-e-vem" (back-and-forth) entre ameaças e contramedidas.

Categoria 1: Ataques ao Protocolo Handshake

- **Objetivo:** Comprometer o Handshake Protocol.
- **Exemplo (1998, [BLEI98]):**
 - Apresentada uma abordagem baseada na exploração da **formatação e implementação** do esquema de criptografia RSA.
- **Evolução do Ataque:**
 - À medida que as contramedidas eram implementadas, o ataque foi **refinado e ajustado** (ex: [BARD12]).
 - **Objetivo do Refinamento:** Não apenas burlar as contramedidas, mas também **acelerar** o tempo do ataque.

Categoria 2: Ataques aos Protocolos de Registro e Aplicação (BEAST)

- Diversas vulnerabilidades foram descobertas nestes protocolos, levando a *patches* (correções).
- **Exemplo 1: BEAST (2011)**
 - (Browser Exploit Against SSL/TLS) - por Thai Duong e Juliano Rizzo [GOOD11].
 - Transformou o que era considerado apenas uma vulnerabilidade **teórica** em um ataque **prático**.
 - **Tipo de Ataque:** *Chosen-Plaintext Attack* (Ataque de Texto Plano Escolhido).
 - (O atacante monta o ataque escolhendo um "palpite" (guess) para o texto plano associado a um texto cifrado conhecido).
 - **Contramedida:** Patches subsequentes foram capazes de bloquear este ataque.

Categoria 2: Ataques (Continuação - CRIME)

- **Exemplo 2: CRIME (2012)**

- (Compression Ratio Info-leak Made Easy) - pelos mesmos autores do BEAST [GOOD12b].
- **Como Funciona:** Explora uma vulnerabilidade quando a **compressão de dados** é usada juntamente com o TLS.
- **Resultado:** Permite ao atacante recuperar o conteúdo de *web cookies*.
- **Ameaça Principal:** Se usado para recuperar cookies de autenticação secretos, permite ao atacante realizar **sequestro de sessão (session hijacking)**.

Categoria 3: Ataques à PKI (Infraestrutura de Chave Pública)

- **Vulnerabilidade:** A atividade de verificar a **validade de certificados X.509** está sujeita a ataques.
- **Exemplo (2012, [GEOR12]):**
 - Demonstrou que bibliotecas SSL/TLS populares sofriam de implementações **vulneráveis de validação de certificados**.
 - Foram reveladas fraquezas no código-fonte de:
 - OpenSSL
 - GnuTLS
 - JSSE (Java)
 - ApacheHttpClient
 - cURL
 - PHP, Python
 - E aplicações construídas sobre elas.

Categoria 4: Outros Ataques (Ex: DoS)

- Refere-se a ataques que não se encaixam nas categorias anteriores [MEYE13].
- **Exemplo: Ataque de Negação de Serviço (DoS) (2011)**
 - Anunciado pelo grupo hacker "The Hackers Choice" [KUMA11].
 - **Como Funciona:** Cria uma carga de processamento pesada no servidor, **sobrecarregando-o com requisições de handshake SSL/TLS**.
 - (Usando o estabelecimento de novas conexões ou a "renegociação" (renegotiation)).
 - **Por que Funciona (Assimetria):** A maioria da computação (CPU) durante um handshake é feita pelo **servidor**, não pelo cliente.
 - **Resultado:** O servidor é forçado a computar continuamente números aleatórios e chaves, levando à exaustão de recursos (DoS).

Ataque: Heartbleed (2014)

- Uma das vulnerabilidades TLS potencialmente **mais catastróficas** já descobertas.
- **Onde:** Encontrado na popular biblioteca *open-source* **OpenSSL**.
- **O Quê:** Um bug (falha) na implementação do **Protocolo Heartbeat** (RFC 6250).
- **Ponto Crítico:**
 - **NÃO** foi uma falha de design (design flaw) na especificação do TLS ou do Heartbeat.
 - Foi um **erro de programação** (programming mistake) específico da biblioteca OpenSSL.

Heartbleed: A Natureza da Vulnerabilidade

- **Revisão do Heartbeat (Comportamento Esperado):**
 - O Cliente envia um `heartbeat_request`.
 - O *request* contém um campo **Payload Length** (L) e um **Payload** (P).
 - O Servidor deve responder com uma **cópia exata** do Payload (P).
- **A Falha de Programação (O Bug):**
 - O software OpenSSL (versão vulnerável) **NÃO CHECAVA** se o tamanho real (bytes) do *payload* recebido era igual ao valor no campo *Payload Length*.
- **O Vetor de Ataque (Exploit):**
 - Um adversário envia um `heartbeat_request` malicioso:
 - Define o **Payload Length** = 64 KB (o máximo).
 - Envia um **Payload** real de apenas 16 bytes (o mínimo).

Heartbleed: O Mecanismo do Vazamento (O Exploit)

- **1. Alocação (No Servidor):**

- O servidor (vulnerável) recebe o *request*.
- Ele lê o **Payload Length (64 KB)** e aloca um *buffer* de memória deste tamanho (64 KB).

- **2. Cópia (O Bug):**

- O servidor copia o *payload* recebido (os **16 bytes** reais) para o início deste buffer.
- **O Problema:** Os 63.9 KB restantes do buffer **não são sobrescritos** (not overwritten).
- Este espaço (quase 64 KB) contém o que quer que estivesse na memória RAM do servidor naquele momento (lixo de memória).

- **3. A Resposta (O Vazamento):**

- O servidor, obedecendo o protocolo, prepara uma *heartbeat_response*.
- Ele lê o **Payload Length (64 KB)** do *request* original e envia de volta **64 KB** de dados do buffer.
- **Resultado:** O atacante recebe seus 16 bytes + 63.9 KB de **dados da memória do servidor**.

Heartbleed: O Impacto Catastrófico

- **Uma "Falha Espetacular":**
 - Ataques repetidos resultam na exposição de grandes quantidades de memória do sistema vulnerável.
- **O que a memória (não sobrescrita) poderia conter?**
 - **Chaves Privadas** (a "joia da coroa" de um servidor SSL).
 - Informações de identificação de usuário.
 - Dados de autenticação (cookies de sessão).
 - Senhas, e outros dados sensíveis.
- **A Tempestade Perfeita:**
 - **Longo Período de Exposição:** O bug não foi descoberto por vários anos.
 - **Ataque Fácil:** O *exploit* é trivial de implementar.
 - **Sem Vestígios:** O ataque não deixa rastros (logs) de que ocorreu.
- **A Escala do Problema:**
 - OpenSSL era a implementação de TLS **mais usada** no mundo.
 - Afetou finanças, bancos, e-mail, redes sociais e governos.
 - Estimativa (na época): Mais de **dois terços (2/3)** dos servidores Web da Internet usavam OpenSSL [GOOD14].

Aula 14 - Firewalls

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
November 27, 2025

Firewalls e VPNs

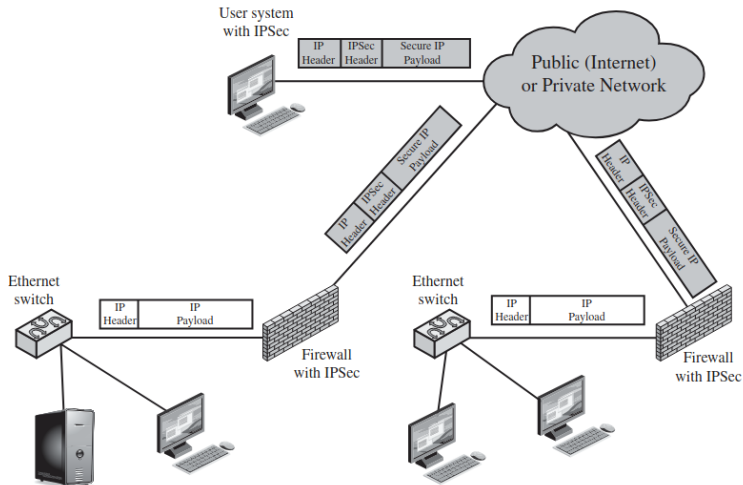


Figure 9.3 A VPN Security Scenario

O que são Firewalls

- Firewalls são um meio **eficaz** de proteger um sistema local ou uma rede contra ameaças de segurança baseadas em rede.
- **O Dilema (O Balanço):**
 - **Proteger** os ativos internos contra ameaças.
 - Ao mesmo tempo, **permitir** o acesso ao mundo exterior (WANs e Internet).
- **9.1 A Necessidade de Firewalls**
 - A necessidade de firewalls é resultado direto da **evolução** constante dos sistemas de informação em corporações, agências governamentais e outras organizações.

A Necessidade de Firewalls: Evolução dos Sistemas (Parte 1)

- Desenvolvimentos notáveis na evolução dos sistemas:
- **1. Sistema Centralizado (Mainframe)**
 - Um mainframe central suportando terminais diretamente conectados.
- **2. Redes Locais (LANs)**
 - Interconectando PCs e terminais entre si e ao mainframe.
- **3. Rede Local Corporativa (Premises Network)**
 - Consiste em várias LANs, interconectando PCs, servidores e (talvez) mainframes.

A Necessidade de Firewalls: Evolução dos Sistemas (Parte 2)

- Desenvolvimentos notáveis (continuação):
- **4. Rede Corporativa Ampla (Enterprise-wide)**
 - Múltiplas redes locais (premises) geograficamente distribuídas.
 - Interconectadas por uma **WAN privada**.
- **5. Conectividade com a Internet**
 - As várias redes locais (premises) se conectam à Internet (além da WAN privada).
- **6. Computação em Nuvem (Enterprise Cloud)**
 - Servidores virtualizados em data centers.
 - Provendo serviços internos (organizacionais) e externos (acessíveis pela Internet).

O Dilema da Conectividade com a Internet

- **A Realidade Atual:**

- A conectividade com a Internet **não é mais opcional** para a maioria das organizações.
- A informação e os serviços disponíveis são essenciais.
- Usuários internos (indivíduos) precisam e desejam o acesso à Internet.

- **O Problema (A Ameaça):**

- Embora o acesso à Internet traga benefícios, ele **permite ao mundo exterior** alcançar e interagir com os ativos da rede local.
- Isso cria uma ameaça significativa para a organização.

- **Por que a Segurança por Máquina não é Suficiente?**

- É possível equipar cada workstation e servidor com segurança forte (ex: proteção contra intrusão).
- **Porém:** Isso pode **não ser suficiente** e, em alguns casos, **não é custo-efetivo**.

- **Conclusão:** A necessidade de uma solução de perímetro (Firewall).

O Desafio da Segurança Apenas no Hospedeiro (Host-Based)

- Considere uma rede com centenas ou milhares de sistemas.
 - Executando vários sistemas operacionais (diferentes versões de Windows, MacOS, Linux).
- **O Problema (Desafio de Escala):**
 - Quando uma falha de segurança é descoberta, **cada** sistema potencialmente afetado deve ser atualizado (patched).
 - Isso requer um gerenciamento de configuração escalável e *patching* agressivo para funcionar.
 - É difícil, embora possível e necessário se for a *única* defesa.
- **A Alternativa (ou Complemento): O Firewall**
 - Uma alternativa amplamente aceita para os serviços de segurança baseados em host.

Firewall: A Defesa de Perímetro

- **Posicionamento:**

- Inserido **entre** a rede local (premises) e a Internet.

- **Objetivo Principal:**

- Estabelecer um **link controlado**.
- Erguer um "muro" de segurança externo, ou **perímetro**.
- Proteger a rede local de ataques baseados na Internet.

- **O Conceito de "Ponto de Gargalo" (Choke Point)**

- Fornece um **ponto único** onde a segurança e a auditoria podem ser impostas.

- **Implementação:**

- Pode ser um único sistema de computador.
- Ou um conjunto de dois ou mais sistemas que cooperam.

- **Resultado: Defesa em Profundidade**

- O firewall fornece uma **camada adicional de defesa**, isolando os sistemas internos.
- Segue a doutrina militar clássica de "defesa em profundidade" (defense in depth).

Objetivos de Design do Firewall (Segundo [BELL94])

- **1. Centralização do Tráfego:**

- **Todo** o tráfego (de dentro para fora e vice-versa) **deve passar** através do firewall.
- *Como isso é alcançado?* Bloqueando fisicamente todo o acesso à rede local, **exceto** pela via do firewall.

- **2. Controle de Autorização:**

- **Apenas** o tráfego autorizado, conforme definido pela **política de segurança local**, terá permissão para passar.

- **3. Imunidade do Próprio Firewall:**

- O próprio firewall deve ser **imune à penetração**.
- *Como isso é alcançado?* Implica o uso de um *hardened system* (sistema reforçado) com um sistema operacional seguro.

O Papel Crítico da Política de Acesso

- Um componente crítico no planejamento e implementação de um firewall é a especificação de uma **Política de Acesso** (Access Policy) adequada.
- **O que é a Política de Acesso?**
 - Lista os tipos de tráfego **autorizados** a passar pelo firewall.
 - Define critérios baseados em:
 - Faixas de endereço (Address ranges)
 - Protocolos
 - Aplicações
 - Tipos de conteúdo (Content types)
- **Desenvolvimento da Política:**
 - **Origem:** Deve ser desenvolvida a partir da **avaliação de risco** (risk assessment) e da política de segurança da informação da organização.
 - **Processo:** Inicia com uma especificação ampla (quais tipos de tráfego a organização *precisa* suportar).
 - **Refinamento:** É então refinada para detalhar os **elementos de filtro** (filter elements) específicos.
 - **Implementação:** Finalmente, é implementada na topologia de firewall apropriada.

Características da Política de Acesso (NIST SP 800-41) - Parte 1

- O NIST SP 800-41 lista características que a política de acesso pode usar para filtrar tráfego:
- **Valores de Endereço IP e Protocolo**
 - Controla o acesso com base em: endereços de origem/destino, números de porta, direção do fluxo (inbound/outbound).
 - Usado por: **Filtros de Pacotes** e **Stateful Inspection Firewalls**.
 - Objetivo: Limitar o acesso a serviços específicos.
- **Protocolo da Aplicação**
 - Controla o acesso com base nos dados autorizados do protocolo da aplicação.
 - Usado por: **Application-Level Gateway** (Proxy).
 - Ex: Checar e-mail (SMTP) para spam, ou requisições (HTTP) para sites autorizados.

- **Identidade do Usuário (User Identity)**
 - Controla o acesso com base na identidade do usuário.
 - Típico para usuários internos (*inside users*).
 - Requer autenticação segura (ex: IPsec).
- **Atividade da Rede (Network Activity)**
 - Controla o acesso com base em considerações como:
 - **Hora da requisição** (ex: apenas em horário comercial).
 - **Taxa de requisições** (ex: para detectar tentativas de *scanning*).
 - Outros padrões de atividade.

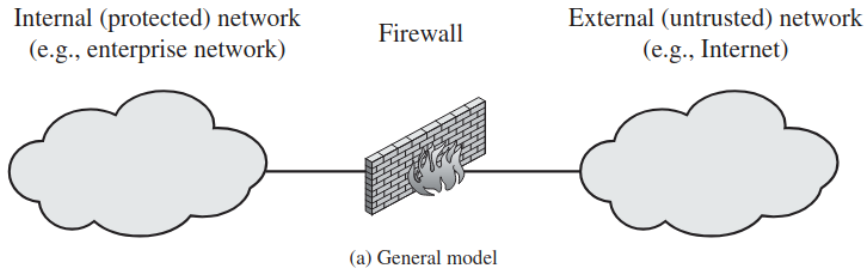
Capacidades do Firewall: O que Esperar

- **1. Ponto de Gargalo Único (Single Choke Point)**
 - Tenta manter usuários não autorizados fora da rede.
 - Proíbe serviços vulneráveis de entrar ou sair da rede.
 - Fornece proteção contra IP spoofing e ataques de roteamento.
 - *Vantagem:* Simplifica a gestão de segurança (capacidades consolidadas).
- **2. Localização para Monitoramento**
 - Permite a implementação de auditorias (audits) e alarmes.
- **3. Plataforma para Funções de Internet (Não-Segurança)**
 - **NAT (Network Address Translator):** Mapeia endereços locais para endereços de Internet.
 - **Gerenciamento de Rede:** Audita ou registra (logs) o uso da Internet.
- **4. Plataforma para IPsec**
 - Pode ser usado para implementar Redes Privadas Virtuais (VPNs).
 - Utiliza a capacidade do "modo túnel" (tunnel mode) do IPsec.

Limitações dos Firewalls

- **1. Não protege contra ataques que "contornam" (bypass) o firewall.**
 - Sistemas internos podem ter acesso direto a um ISP (ex: banda larga móvel ou com fio).
 - LANs internas podem ter conexões diretas com organizações parceiras.
- **2. Não protege totalmente contra ameaças internas.**
 - Ex: Um funcionário insatisfeito (disgruntled employee).
 - Ex: Um funcionário que coopera (sem saber) com um atacante externo (ex: phishing).
- **3. LANs sem fio (Wireless) inseguras.**
 - Podem ser acessadas de fora da organização.
 - Firewalls internos (separando redes) não podem proteger contra comunicação wireless direta entre sistemas em lados opostos do firewall.
- **4. Dispositivos portáteis infectados (BYOD).**
 - Um laptop, PDA ou dispositivo de armazenamento portátil (pen drive) pode ser usado e infectado **fora** da rede corporativa.
 - E, em seguida, ser conectado e usado **internamente**, transportando a ameaça.

Firewall genérico



Tipos de Firewalls

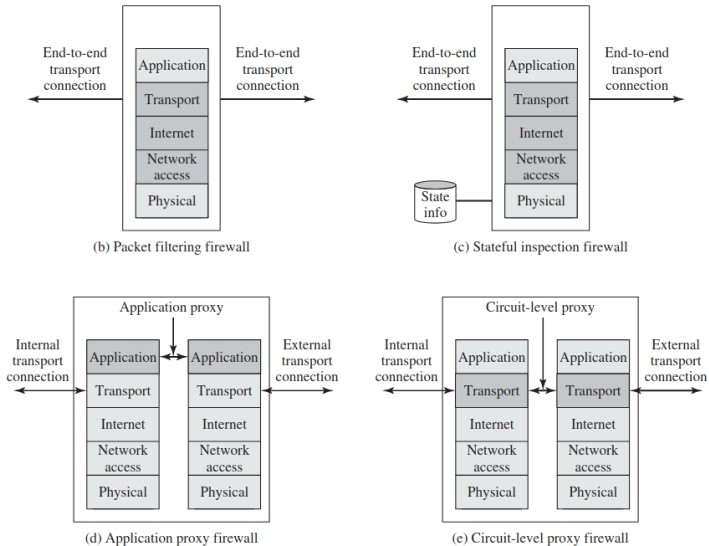


Figure 9.1 Types of Firewalls

Introdução: Tipos de Firewalls

- Um firewall pode monitorar o tráfego de rede em **vários níveis**:
 - Pacotes de rede de baixo nível (individualmente ou como um fluxo).
 - Todo o tráfego dentro de uma conexão de transporte.
 - Detalhes dos protocolos de aplicação.
- O nível de monitoramento apropriado é determinado pela **política de acesso** (access policy) desejada.
- **Lógica de Filtragem (Implementa a Política):**
 - **Filtro Positivo (Positive Filter):** Permite passar *apenas* os pacotes que atendem a critérios específicos.
 - **Filtro Negativo (Negative Filter):** Rejeita *qualquer* pacote que atenda a certos critérios.
- **O que o Firewall Examina (Depende do Tipo):**
 - Um ou mais cabeçalhos de protocolo em cada pacote.
 - O *payload* (carga útil) de cada pacote.
 - O padrão gerado por uma **sequência** de pacotes.

Tipo 1: Firewall de Filtro de Pacotes (Definição)

- Chamado de *Packet Filtering Firewall*.
- Aplica um conjunto de regras a **cada** pacote IP (de entrada e saída).
- Ação resultante: **Encaminhar** (forward) ou **Descartar** (discard) o pacote.
- Tipicamente configurado para filtrar pacotes em **ambas as direções** (de e para a rede interna).
- **Regras baseadas nas informações do pacote (Camadas 3 e 4):**
 - **Endereço IP de Origem:** O IP do sistema que originou o pacote.
 - **Endereço IP de Destino:** O IP do sistema que o pacote tenta alcançar.
 - **Porta de Origem/Destino (Nível Transporte):** O número da porta TCP ou UDP (que define aplicações como SNMP ou HTTP).
 - **Campo Protocolo IP:** Define o protocolo de transporte (ex: TCP, UDP, ICMP).
 - **Interface:** Em firewalls com 3+ portas, indica de qual interface o pacote veio ou para qual vai.

Filtro de Pacotes: Lógica de Regras e Políticas Padrão

- O filtro é configurado como uma **lista de regras**.
- O firewall compara os cabeçalhos (IP/TCP) de cada pacote com esta lista.
- **Se houver correspondência:** A regra é invocada (encaminhar/descartar).
- **Se NÃO houver correspondência:** A **ação padrão** (default action) é tomada.
- **Duas Políticas Padrão Possíveis:**
 - **1. Padrão = Descartar (Default Discard)**
 - "Aquilo que não é expressamente permitido é proibido."
 - **Mais conservadora (segura).**
 - Inicialmente, tudo é bloqueado; serviços são adicionados caso a caso.
 - Mais visível para usuários (visto como um obstáculo no início).
 - Preferida por empresas e governo.
 - **2. Padrão = Encaminhar (Default Forward)**
 - "Aquilo que não é expressamente proibido é permitido."
 - Aumenta a facilidade de uso para os usuários finais.
 - **Segurança reduzida.**
 - O admin deve "reagir" a cada nova ameaça conforme ela se torna conhecida.
 - Usada por organizações mais abertas (ex: universidades).

Exemplo de regras de filtragem em Firewalls

Table 9.1 Packet-Filtering Examples

Rule	Direction	Src address	Dest addresss	Protocol	Dest port	Action
1	In	External	Internal	TCP	25	Permit
2	Out	Internal	External	TCP	>1023	Permit
3	Out	Internal	External	TCP	25	Permit
4	In	External	Internal	TCP	>1023	Permit
5	Either	Any	Any	Any	Any	Deny

Filtro de Pacotes: Exemplo de Regras SMTP (Tabela 9.1)

- **Objetivo:** Permitir tráfego de e-mail (SMTP, porta 25) de entrada e saída, mas bloquear todo o resto.
- **Lógica:** Regras aplicadas de cima para baixo (top-to-bottom).
- **Intenção das Regras:**
 - **Regra 1:** Permitir e-mail **de entrada** (destino porta 25).
 - **Regra 2:** Permitir a **resposta** ao e-mail de entrada.
 - **Regra 3:** Permitir e-mail **de saída** (destino porta 25).
 - **Regra 4:** Permitir a **resposta** ao e-mail de saída.
 - **Regra 5:** Política padrão (Default = Discard). (Sempre a última regra, explícita ou implicitamente).
- **O Problema (A Falha):**
 - **A Regra 4 é muito permissiva.** Ela permite tráfego externo para **qualquer** porta de destino acima de 1023 (assumindo que seja uma "resposta").

Exemplo SMTP: O Exploit e a Solução

- **Exemplo de Exploit (usando a Regra 4):**

- Um atacante externo abre uma conexão da porta 5150 (do atacante) para um Web Proxy interno na porta 8080.
- A Regra 4 (antiga) **permite** isso, pois a porta de destino (8080) é 1023.
- Isso deveria ser proibido e poderia permitir um ataque ao servidor proxy.

- **A Solução (Refinando as Regras):**

- Configurar o campo de **porta de origem (source port)** nas regras, tornando-as mais específicas (menos permissivas).

- **Regras Refinadas:**

- **Regras 2 e 4 (Respostas):**

- Definir a Porta de Origem = **25**.
- (Pois a resposta a um e-mail deve vir da porta 25 do servidor).

- **Regras 1 e 3 (Novas conexões):**

- Definir a Porta de Origem = **1023**.
- (Pois um cliente iniciando um e-mail usará uma porta alta).

Filtro de Pacotes: Vulnerabilidade Remanescente (SMTP)

- Mesmo com as regras de porta de origem refinadas, uma vulnerabilidade permanece.
- **A Situação (Regras 3 e 4):**
 - Permitem que qualquer host interno envie e-mail para fora (Porta de Destino = 25).
- **O Problema:**
 - O uso da porta 25 para recebimento de SMTP é apenas um **padrão** (default).
- **O Risco (O Exploit):**
 - Uma máquina externa (atacante) poderia ser configurada para rodar **outra aplicação** (maliciosa) na porta 25.
 - O atacante poderia então enviar pacotes **DA** sua porta de origem 25 (TCP Source Port = 25) para tentar invadir máquinas internas.
 - A Regra 4 (revisada) permitiria isso, pois ela está configurada para aceitar pacotes *vindos* da porta 25 (assumindo ser uma resposta).

Filtro de Pacotes: A Solução (Flag ACK)

- **A Contramedida:**

- Para combater essa ameaça, podemos adicionar um campo de **Flag ACK** (ACK flag) à verificação da regra.

- **Onde aplicar:**

- Esta verificação é adicionada à **Regra 4** (a regra que permite a *resposta* do servidor externo).

- **A Nova Lógica (Regra 4):**

- A regra agora exigiria que o **flag ACK** esteja **definido** (set) no pacote de entrada.

- **Por que isso funciona?**

- Um pacote que é parte de uma conexão TCP **estabelecida** (como uma resposta SMTP legítima) **SEMPRE** terá o flag ACK definido.
- Um pacote de um atacante tentando **iniciar** uma nova conexão (o exploit) **NÃO** teria o flag ACK definido (ex: teria um flag SYN).

- A Regra 4 agora se pareceria com:

- (O texto a seguir deve descrever a nova regra, incluindo a verificação do Flag ACK).

Exemplo de regras de filtragem em Firewalls

Rule	Direction	Src address	Src port	Dest address	Protocol	Dest port	Flag	Action
4	In	External	25	Internal	TCP	>1023	ACK	Permit

A Lógica da Nova Regra:

- A regra agora permite pacotes de entrada (respostas) que possuem:
 - Porta de Origem (Source Port) = 25
 - **E** o flag ACK definido no segmento TCP.
- (Isso efetivamente bloqueia pacotes que *iniciam* conexões (ex: flag SYN) da porta 25, que era a vulnerabilidade.)

Filtro de Pacotes: Vantagens

- **1. Simplicidade**
 - A sua lógica de regras é direta (baseada em cabeçalhos L3/L4).
- **2. Transparência**
 - Geralmente são transparentes para os usuários (não exigem configuração no cliente).
- **3. Velocidade**
 - São muito rápidos, pois não inspecionam o payload (carga útil) do pacote.
- **Desvantagens:**
 - O NIST SP 800-41 lista várias fraquezas significativas.

- **1. Não examina dados da camada superior (Payload)**
 - **Problema:** Não pode prevenir ataques que usam vulnerabilidades **específicas da aplicação**.
 - **Exemplo:** Não pode bloquear comandos de aplicação específicos (se o firewall permite a aplicação, todas as funções dela são permitidas).
- **2. Funcionalidade de Log Limitada**
 - **Problema:** Devido à informação limitada que o firewall vê.
 - **Conteúdo do Log:** Geralmente contém apenas as informações usadas para a decisão (IPs de origem/destino, tipo de tráfego).

- **3. Não suporta autenticação avançada de usuário**
 - **Problema:** Novamente, devido à falta de funcionalidade de camada superior.
- **4. Vulnerável a ataques contra a pilha TCP/IP**
 - **Exemplo:** *Network layer address spoofing* (falsificação de endereço IP).
 - **Problema:** Muitos filtros não conseguem detectar um pacote onde a informação de endereçamento (Camada 3) foi alterada.
 - **Uso:** Usado por intrusos para "burlar" (bypass) os controles do firewall.
- **5. Suscetível a configurações incorretas (Misconfiguration)**
 - **Problema:** Devido ao pequeno número de variáveis usadas nas decisões.
 - **Risco:** É fácil configurar acidentalmente o firewall para permitir tráfego que deveria ser negado.

Ataques ao Filtro de Pacotes: IP Address Spoofing

- **O Ataque (IP Address Spoofing):**
 - O intruso transmite pacotes **de fora** da rede.
 - O campo "Source IP Address" (IP de Origem) é falsificado (spoofed) para conter o endereço de um **host interno** confiável.
- **O Objetivo:**
 - Penetrar sistemas que empregam segurança simples baseada no endereço de origem (ex: "aceitar pacotes vindos de IPs internos confiáveis").
- **A Contramedida:**
 - O firewall (ou roteador externo) deve **descartar** pacotes se:
 - O pacote chega na interface **externa**.
 - **E** o pacote possui um endereço de origem (source address) **interno**.

Ataques ao Filtro de Pacotes: Tiny Fragment

- **O Ataque (Tiny Fragment Attack):**
 - O intruso usa a opção de fragmentação IP para criar **fragmentos extremamente pequenos**.
 - **Objetivo:** Forçar o cabeçalho TCP (que contém as portas) para um **fragmento separado** (ex: o segundo fragmento), deixando o primeiro fragmento apenas com o cabeçalho IP.
- **A Lógica do Ataque:**
 - O filtro de pacotes toma sua decisão (Permitir/Negar) com base no **primeiro fragmento**.
 - O atacante espera que o primeiro fragmento (incompleto) seja permitido, e que os fragmentos subsequentes (com o TCP malicioso) passem direto.
- **A Contramedida:**
 - Reforçar uma regra de que o **primeiro fragmento** de um pacote **deve conter** uma quantidade mínima predefinida do cabeçalho de transporte (TCP).
 - (Se o primeiro fragmento for rejeitado, o filtro deve memorizar o ID do pacote e descartar todos os fragmentos subsequentes).

O Problema do Filtro de Pacotes (Falta de Contexto)

- Um filtro de pacotes tradicional (stateless) toma decisões **pacote a pacote**.
- Ele **não considera o contexto** de camada superior (se uma conexão TCP existe).
- **Contexto (Modelo Cliente/Servidor):**
 - Aplicações (ex: SMTP) usam TCP.
 - **Servidor (ex: SMTP):** Usa uma "porta bem conhecida" (well-known), que é fixa e ≤ 1024 (ex: Porta **25**).
 - **Cliente (ex: usuário):** Usa uma porta dinâmica, temporária e > 1024 (ex: Porta **49152**).
- **A Vulnerabilidade do Filtro de Pacotes:**
 - Para permitir o tráfego de *resposta* (do servidor na porta 25 para o cliente na porta 49152), o firewall deve permitir tráfego de entrada em **TODAS** as portas altas (> 1023).
 - Isso cria uma vulnerabilidade enorme que pode ser explorada por atacantes.

Tipo 2: Firewall de Inspeção de Estado (Stateful)

- (Stateful Packet Inspection Firewall).
- **A Solução:** "Aperta" as regras para o tráfego TCP.
- **O Mecanismo (O "Estado"):**
 - O firewall cria e mantém um **diretório (Tabela de Estado)** das conexões TCP de *saída* (outbound) que estão ativas.
 - (Ref: Tabela 9.2).
 - Uma entrada é criada para cada conexão atualmente estabelecida.
- **A Nova Lógica de Filtragem:**
 - O firewall agora permite tráfego de entrada em portas altas (>1023) **APENAS** se este pacote **corresponder** (fit the profile) a uma das entradas (conexões) existentes no diretório (Tabela de Estado).

Firewall de Inspeção de Estado (Capacidades)

- Um firewall stateful revisa as mesmas informações que um filtro de pacotes (cabeçalhos L3/L4).
- **MAS TAMBÉM...**
 - Registra informações sobre o **estado da conexão TCP** (ex: estabelecida, fechando, etc.).
- **Recursos Avançados (em alguns firewalls stateful):**
 - **Rastreamento de Números de Sequência TCP:**
 - Usado para prevenir ataques que dependem do número de sequência (ex: *session hijacking*).
 - **Inspeção Limitada de Aplicação (Deep Packet Inspection - DPI):**
 - Inspeccionam (de forma limitada) os dados da aplicação.
 - Útil para protocolos "problemáticos" (ex: FTP, IM, SIPs) para identificar e rastrear conexões de dados relacionadas.

Filtro de pacotes com estado

Table 9.2 Example Stateful Firewall Connection State Table

Source Address	Source Port	Destination Address	Destination Port	Connection State
192.168.1.100	1030	210.9.88.29	80	Established
192.168.1.102	1031	216.32.42.123	80	Established
192.168.1.101	1033	173.66.32.122	25	Established
192.168.1.106	1035	177.231.32.12	79	Established
223.43.21.231	1990	192.168.1.6	80	Established
219.22.123.32	2112	192.168.1.6	80	Established
210.99.212.18	3321	192.168.1.6	80	Established
24.102.32.23	1025	192.168.1.6	80	Established
223.21.22.12	1046	192.168.1.6	80	Established

Tipo 3: Application-Level Gateway (Proxy de Aplicação)

- Também chamado de **Application Proxy**.
- Atua como um **"relay" (retransmissor)** do tráfego em nível de aplicação (ex: Telnet, FTP).
- **Mecanismo de "Dupla Conexão" (Splice):**
 - O usuário contata o gateway (proxy).
 - O gateway solicita ao usuário o host remoto e as credenciais (ID/autenticação).
 - **Se válido:** O gateway (ele mesmo) contata a aplicação no host remoto.
 - O gateway retransmite (relays) os segmentos TCP (dados da aplicação) entre os dois *endpoints*.
 - (Existem duas conexões "emendadas" (spliced): Cliente ↔ Gateway e Gateway ↔ Servidor).

Application-Level Gateway: Vantagens e Desvantagens

- **Controle Granular:**
 - Se o gateway não implementa o código proxy para uma aplicação, o serviço **não é suportado**.
 - Pode ser configurado para suportar apenas **recursos específicos** de uma aplicação (ex: permitir GET no HTTP, mas negar POST) que o admin considera aceitáveis.
- **Vantagem: Mais Seguro que Filtros de Pacote**
 - Precisa apenas analisar **algumas aplicações permitidas**, em vez de inúmeras combinações complexas de IP/Porta/Flags.
- **Vantagem: Auditoria (Logging)**
 - É fácil registrar e auditar todo o tráfego no nível da aplicação.
- **Desvantagem: Sobrecarga de Processamento (Overhead)**
 - O gateway deve examinar e encaminhar todo o tráfego nas duas conexões "emendadas".

Tipo 4: Circuit-Level Gateway (Proxy de Circuito)

- Pode ser um sistema autônomo (stand-alone) ou uma **função especializada** de um Application-Level Gateway.
- **Mecanismo (Similar ao Proxy de Aplicação):**
 - **NÃO** permite uma conexão TCP ponta-a-ponta (end-to-end).
 - O gateway estabelece **duas conexões TCP**:
 - 1. (Host Interno ↔ Gateway)
 - 2. (Gateway ↔ Host Externo)
- **Diferença Chave (vs. Application Proxy):**
 - Uma vez estabelecidas as conexões, o gateway retransmite (relays) os segmentos TCP **sem examinar o conteúdo (payload)**.
 - A **função de segurança** consiste *apenas* em determinar quais conexões serão permitidas.
- **Caso de Uso Típico (Híbrido):**
 - Quando o admin **confia** nos usuários internos.
 - *Inbound (Entrada)*: Usa-se Application Proxy (caro, mas seguro).
 - *Outbound (Saída)*: Usa-se Circuit-Level (barato), pois não se gasta processamento examinando os dados de saída.

Exemplo de Circuit-Level: SOCKS (RFC 1928)

- SOCKS (versão 5, RFC 1928) é uma implementação de um Circuit-Level Gateway.
- **Definição (RFC):** Um framework para aplicações cliente-servidor (TCP e UDP) usarem os serviços de um firewall de forma conveniente e segura.
- **Posição na Pilha:**
 - É conceitualmente uma "camada de ajuste" (*shim-layer*).
 - Fica **entre** a camada de Aplicação e a camada de Transporte.
 - (Não é um gateway de camada de Rede; não encaminha ICMP).
- **Componentes do SOCKS:**
 - **SOCKS Server:** Roda no firewall (ex: UNIX, Windows).
 - **SOCKS Client Library:** Roda nos hosts internos (protegidos).
 - **Clientes "SOCKS-ificados":** Programas (FTP, TELNET) recompilados ou "relinkados" para usar a biblioteca SOCKS.

SOCKS: Mecanismo de Operação (TCP e UDP)

- **Fluxo de Conexão TCP:**

- 1. O cliente (interno) abre uma conexão TCP com o **Servidor SOCKS** (na porta TCP **1080**).
- 2. O cliente negocia o método de autenticação.
- 3. O cliente se autentica.
- 4. O cliente envia uma **solicitação de relay** (relay request) (ex: "quero conectar no IP 1.2.3.4, porta 80").
- 5. O Servidor SOCKS avalia o pedido e (se permitido) estabelece a conexão externa.

- **Tratamento de UDP (Similar):**

- Uma conexão **TCP** é aberta primeiro (para o SOCKS, porta 1080) apenas para **autenticar** o usuário.
- Os segmentos UDP são então encaminhados (relay) pelo firewall *enquanto* essa conexão TCP de controle permanecer aberta.

9.4 Plataformas de Firewall (Firewall Basing)

- É comum instalar um firewall em uma **máquina "stand-alone"** (dedicada).
 - Rodando um S.O. comum (ex: UNIX, Linux).
 - **Ou que** ... Pode ser fornecido como um "security appliance" pré-configurado.
- **Alternativas:**
 - A funcionalidade de firewall também pode ser implementada como um **módulo de software** em:
 - Um Roteador.
 - Um Switch LAN.
 - Um Servidor.
- Esta seção analisa considerações adicionais sobre essas plataformas.

Host Bastião: Definição

- Um **Host Bastião** é um sistema identificado pelo administrador do firewall como um **"ponto forte crítico"** (critical strong point) na segurança da rede.
- **Função Típica:**
 - Serve como plataforma para **Gateways** (de Nível de Aplicação ou Nível de Circuito).
 - Pode suportar outros serviços (ex: IPsec).
- **Característica Principal:**
 - A plataforma de hardware executa uma **versão segura** do seu S.O.
 - Isso o torna um **Sistema Reforçado** (Hardened System).

Características do Host Bastião (Parte 1: Hardening e Acesso)

- **Serviços Mínimos:**

- **Apenas** os serviços que o administrador considera **essenciais** são instalados.
- Ex: Proxies para DNS, FTP, HTTP, SMTP.

- **Autenticação Adicional:**

- O host pode exigir **autenticação extra** antes de permitir o acesso aos serviços de proxy.
- Além disso, cada serviço de proxy (individualmente) pode exigir sua própria autenticação.

Características do Host Bastião (Parte 2: Restrições de Proxy)

- **Subconjunto de Comandos (Command Subset):**
 - Cada proxy é configurado para suportar apenas um **subconjunto** dos comandos padrão da aplicação (ex: bloquear certas funções do HTTP).
- **Acesso Restrito a Hosts:**
 - Cada proxy é configurado para permitir acesso **apenas a sistemas host específicos** (internos).
- **Auditoria Detalhada (Logging):**
 - Cada proxy mantém logs de auditoria detalhados (todo o tráfego, cada conexão, duração).
 - (Ferramenta essencial para descobrir e terminar ataques).

Características do Host Bastião (Parte 3: Design de Software)

- **Simplicidade do Software:**

- Cada proxy é um pacote de software **muito pequeno**, desenhado especificamente para segurança.
- **Vantagem:** Mais fácil de verificar (auditar) em busca de falhas de segurança.
- *Ex: App de e-mail UNIX (20.000+ linhas) vs. Proxy de e-mail (j 1.000 linhas).*

- **Independência (Modularidade):**

- Cada proxy é **independente** dos outros no host.
- **Vantagem:** Se um proxy tiver uma vulnerabilidade, ele pode ser desinstalado sem afetar os outros.
- **Vantagem:** Fácil de adicionar suporte a novos serviços instalando um novo proxy.

Características do Host Bastião (Parte 4: Segurança em Runtime)

- **Sem Acesso a Disco:**

- Um proxy (geralmente) não realiza acesso a disco, exceto para ler seu arquivo de configuração inicial.
- **Vantagem:** As partes do sistema de arquivos com código executável podem ser tornadas **somente-leitura**.
- **Resultado:** Dificulta para um intruso instalar *Trojan horses*, *sniffers* ou outros arquivos.

- **Sem Privilégios:**

- Cada proxy roda como um **usuário não-privilegiado** (nonprivileged user).
- Roda em um diretório privado e seguro no host bastião.

Host-Based Firewalls

- **O que é?**
 - Um **módulo de software** usado para proteger um **host individual**.
 - Disponível no S.O. ou como pacote add-on.
 - Filtra e restringe o fluxo de pacotes (assim como firewalls stand-alone).
 - Localização comum: em um **servidor**.
- **Vantagens:**
 - **1. Regras Personalizadas:**
 - As regras de filtragem podem ser adaptadas ao ambiente específico do host (ex: filtros diferentes para servidores de aplicações diferentes).
 - **2. Proteção Independente da Topologia:**
 - Ataques **internos** e externos devem passar pelo firewall do host.
 - **3. Camada Adicional de Proteção:**
 - Usado *em conjunto* com firewalls stand-alone (defesa em profundidade).
 - Permite adicionar novos servidores à rede (com seus próprios firewalls) sem a necessidade de alterar a configuração do firewall da rede.

Firewalls em Dispositivos de Rede

- **O que são?**
 - Funcionalidades de firewall (especialmente filtro de pacotes e stateful inspection) implementadas diretamente em dispositivos de rede.
- **Onde?**
 - Comumente em **roteadores** e **switches**.
- **Função:**
 - Monitorar e filtrar os fluxos de pacotes que passam *através* do dispositivo.
- **Propósito (Defesa em Profundidade):**
 - Usados para fornecer **camadas adicionais de proteção**, em conjunto com bastion hosts e host-based firewalls.

Virtual Firewalls (Firewalls Virtuais)

- **Contexto: Ambientes Virtualizados**

- Em vez de dispositivos físicos separados (servidores, switches, firewalls), usam-se versões **virtualizadas** que compartilham o mesmo hardware físico.

- **Implementações (Duas Formas):**

- **1. Appliance Virtual:** Pode ser um "bastion host" virtualizado (rodando como uma VM).
- **2. No Hypervisor:** As capacidades de firewall podem ser fornecidas diretamente pelo **hypervisor** (o software que gerencia as máquinas virtuais).

Locais de um firewall

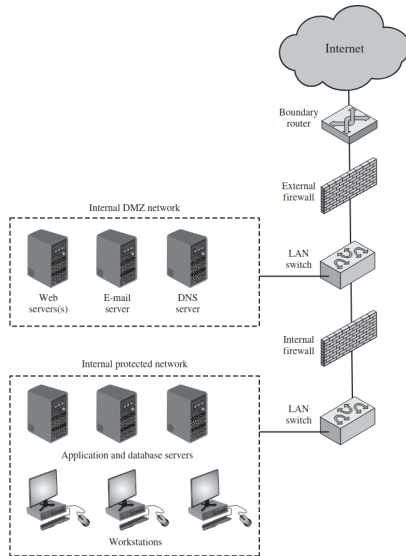


Figure 9.2 Example Firewall Configuration

Firewalls em dispositivos locais: Definição

- **O que é?**
 - Controla o tráfego entre um **computador pessoal** (ou workstation) e a rede (Internet ou rede corporativa).
- **Casos de Uso:**
 - Ambiente doméstico (home).
 - Intranets corporativas.
- **Localização (Implementação):**
 - **1. Módulo de Software:** Tipicamente, roda no próprio computador pessoal.
 - **2. Roteador Doméstico:** Em ambientes com múltiplos computadores, a funcionalidade pode estar no roteador que se conecta ao modem (DSL/Cabo).

Firewalls em dispositivos locais: Função e Implementações

- **Complexidade:**
 - Tipicamente **muito menos complexos** que firewalls de servidor ou stand-alone.
- **Função Principal (Defesa):**
 - Negar acesso remoto **não autorizado** ao computador.
- **Função Secundária (Detecção):**
 - Monitorar atividade **de saída** (outgoing) na tentativa de detectar e bloquear malware (ex: worms).
- **Implementações (Exemplos):**
 - Pacote netfilter (Linux).
 - Pacote pf (BSD e MacOS).
 - Windows Firewall (Windows).
- **Configuração:**
 - Via linha de comando (CLI) ou interface gráfica (GUI).

Firewalls em dispositivos locais: Política Padrão e Serviços

- **Política Padrão (Default):**

- **Conexões de Entrada (Inbound):** Geralmente **negadas**, exceto as que o usuário permite explicitamente.
- **Conexões de Saída (Outbound):** Geralmente **permitidas**.

- **Exemplos de serviços de entrada que podem ser reativados (portas):**

- Personal file sharing (548, 427)
- Windows sharing (139)
- Personal Web sharing (80, 427)
- Remote login—SSH (22)
- FTP access (20-21, etc.)
- Printer sharing (631, 515)
- IChat Rendezvous (5297, 5298)
- iTunes Music Sharing (3869)
- CVS (2401)
- Gnutella/Limewire (6346)
- ICQ (4000)
- IRC (194)
- MSN Messenger (6891-6900)
- Network Time (123)
- SMB (sem netbios—445)
- VNC (5900-5902)

- **Exceção da Regra de FTP:**
 - Se o acesso FTP é habilitado, as portas 20 e 21 são abertas localmente.
 - Se *outros* conectam *a partir* das portas 20 ou 21, as portas 1024-65535 são abertas (para a conexão de dados do FTP).
- **Recursos Avançados:**
 - **Stealth Mode:** "Esconde" o sistema na Internet ao **descartar (drop)** pacotes de comunicação não solicitados (fazendo parecer que o sistema não está presente).
 - **Bloqueio de UDP:** Restringe o tráfego (para portas abertas) apenas a pacotes TCP.
 - **Logging:** Ferramenta importante para verificar atividades indesejadas.
 - **Filtro de Aplicação:** Permite ao usuário especificar que **apenas aplicações selecionadas** (ou assinadas por uma CA válida) podem prover serviços na rede.

9.5 Localização e Configurações do Firewall

- O firewall é posicionado para prover uma **barreira protetora** entre uma fonte externa (potencialmente não confiável) e uma rede interna.
- O administrador de segurança deve decidir:
 - A **localização** do firewall.
 - O **número** de firewalls necessários.
- Esta seção analisa algumas opções comuns de configuração e localização.

Configuração: Redes DMZ (Zona Desmilitarizada)

- Uma configuração de firewall comum que inclui um **segmento de rede adicional** entre um firewall **externo** e um **interno**.
- **Firewall Externo:** Colocado na borda da rede (logo após o roteador de borda da Internet/WAN).
- **Firewall(s) Interno(s):** Protegem o grosso da rede corporativa (enterprise network).
- **A DMZ:** É a região (rede) localizada *entre* esses dois firewalls.
- **Propósito da DMZ:**
 - Localizar sistemas que precisam ser **acessíveis externamente**, mas que ainda necessitam de **alguma proteção**.
 - **Exemplos de sistemas na DMZ:** Website corporativo, Servidor de e-mail (SMTP), Servidor DNS.

DMZ: O Papel de Cada Firewall

- **Função do Firewall Externo:**

- Fornece controle de acesso e proteção (moderada) para os sistemas da DMZ (consistente com sua necessidade de conectividade).
- Fornece um nível básico de proteção para o resto da rede interna.

- **Três Propósitos do Firewall Interno:**

1. **Filtragem mais rigorosa:** Adiciona capacidade de filtragem mais forte (comparado ao externo) para proteger os servidores/workstations internos de ataques externos.
2. **Proteção de Mão Dupla (vs. DMZ):**
 - Protege a rede interna contra ataques lançados *a partir* da DMZ (ex: se um servidor web for comprometido com malware, rootkits, bots).
 - Protege a DMZ contra ataques vindos da rede *interna*.
3. **Segmentação Interna:**
 - Múltiplos firewalls internos podem ser usados para proteger porções da rede interna umas das outras (ex: Servidores vs. Workstations, como na Fig 8.5).

Configuração: Redes Privadas Virtuais (VPNs)

- **O Problema (Custo vs. Segurança):**

- Organizações (com LANs em locais dispersos) precisam se interconectar.
- Usar a **Internet** (rede pública) é mais barato e mais fácil de gerenciar (offloading) do que usar uma **rede privada** (linhas privadas).
- **Mas:** A rede pública expõe o tráfego corporativo (eavesdropping, acesso não autorizado).

- **A Solução (VPN):**

- Um conjunto de computadores que se interconecta por uma rede insegura.
- Usa **criptografia e autenticação** em camadas de protocolo inferiores.
- Cria uma **conexão segura** através de uma rede insegura (Internet).
- *Pré-requisito:* Depende de ter o mesmo sistema de cripto/autenticação em ambas as pontas.

VPNs: Implementação com IPsec

- A criptografia pode ser feita pelo **software do firewall** ou por roteadores.
- Protocolo mais comum: **IPsec** (nível IP).
- **Cenário Típico:**
 - Protocolos IPsec operam em dispositivos de rede (roteador ou firewall) que conectam a LAN ao mundo externo.
 - O dispositivo IPsec **criptografa e comprime** todo o tráfego que *sai* para a WAN.
 - O dispositivo IPsec **descriptografa e descomprime** todo o tráfego que *vem* da WAN.
 - Esta operação é **transparente** para os servidores e workstations na LAN.
- **Usuários Individuais (Remotos):**
 - Também podem usar IPsec, mas o *próprio workstation* deve implementar os protocolos e ter alta segurança de host (tornando-se um alvo atraente).

VPNs: Onde implementar o IPsec?

- Um local lógico para implementar o IPsec é **no firewall**.
- **Problema 1: IPsec atrás (interno) do firewall**
 - O tráfego VPN que passa pelo firewall (em ambas as direções) está **criptografado**.
 - **Consequência:** O firewall é **incapaz** de realizar suas funções (filtragem, scan de vírus, log, controle de acesso, etc.), pois não vê o tráfego em claro.
- **Problema 2: IPsec no roteador de borda (externo ao firewall)**
 - O dispositivo (roteador) é provavelmente **menos seguro** que o firewall.
 - Portanto, é uma plataforma menos desejável para o IPsec.
- *(Solução implícita: A implementação do IPsec no próprio firewall é a mais funcional).*

Configuração: Firewalls Distribuídos

- **O que é?** Uma configuração que envolve:
 - Dispositivos de firewall stand-alone (de rede).
 - **E** Host-Based Firewalls (em servidores, workstations).
- **Característica Chave:** Trabalham juntos sob um **controle administrativo central**.
- **Gerenciamento:**
 - Administradores configuram firewalls em centenas de servidores/workstations e firewalls pessoais (locais e remotos).
 - Ferramentas permitem ao admin definir políticas e monitorar a segurança em **toda a rede**.
- **Vantagem:**
 - Proteção contra **ataques internos**.
 - Proteção **personalizada** (tailored) para máquinas e aplicações específicas.

Configuração: Firewalls Distribuídos

- **O que é?** Uma configuração que envolve:
 - Dispositivos de firewall stand-alone (de rede).
 - **E** Host-Based Firewalls (em servidores, workstations).
- **Característica Chave:** Trabalham juntos sob um **controle administrativo central**.
- **Gerenciamento:**
 - Administradores configuram firewalls em centenas de servidores/workstations e firewalls pessoais (locais e remotos).
 - Ferramentas permitem ao admin definir políticas e monitorar a segurança em **toda a rede**.
- **Vantagem:**
 - Proteção contra **ataques internos**.
 - Proteção **personalizada** (tailored) para máquinas e aplicações específicas.

Aula 15 - IDS/IPS

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
November 27, 2025

A Ameaça: Intrusos e a Necessidade de Defesa

- Uma das principais ameaças à segurança são os **intrusos**.
- Boa parte das violações são feitas por **outsiders**
- Mas existem algumas violações feitas por **insiders**.
- **Porém:** Insiders podem ser muito mais perigosos.
- **Preocupação:** Ataques **direcionados** a indivíduos e seus sistemas de TI.
- **Implicação:**
 - Ataques direcionados podem *burlar* as defesas de perímetro (ex: Firewalls).
 - É necessário estratégias de **Defesa em Profundidade**.

Classes de Intrusos (Parte 1): Ciber criminosos e Ativistas

Entender a **motivação** dos atacantes auxilia no design da estratégia defensiva.

- **Classe 1: Ciber criminosos**

- **Motivação:** Recompensa Financeira.
- **Atividades:** Roubo de identidade, roubo de credenciais financeiras, espionagem corporativa, roubo ou sequestro de dados.
- **Perfil:** Organizam-se em fóruns *underground* (ex: DarkMarket) para trocar dados e coordenar ataques.

- **Classe 2: Ativistas**

- **Motivação:** Causas sociais ou políticas.
- **Nível de Habilidade:** Frequentemente baixo.
- **Atividades:** Promover a causa através de *website defacement*, DoS, ou vazamento de dados.
- **Exemplos:** Anonymous, LulzSec, Manning, Snowden.

Classes de Intrusos (Parte 2): APTs e Outros

- **Classe 3: APTs**

- **Quem são?** Grupos de hackers **patrocinados por governos**.
- **Motivação:** Conduzir **espionagem** ou **sabotagem**.
- **Alias: APTs (Advanced Persistent Threats)** – devido à natureza **secreta** e **persistência** por longos períodos.
- **Escopo:** Generalizado (China, Rússia, EUA, UK, etc.).

- **Classe 4: Outros**

- **Hackers Clássicos:**

- **Motivação:** Desafio técnico, ou estima e reputação no grupo.
- Ex: Responsáveis por descobrir novas vulnerabilidades, como buffer overflow.

- **Hobby Hackers:**

- Usam *attack toolkits* (ferramentas prontas) para explorar.
- Potencialmente podem ser recrutados pelas outras classes.

Exemplos de Intrusão (NIST SP 800-61) - Parte 1

O NIST SP 800-61 (Guia de Resposta a Incidentes) lista os seguintes exemplos de intrusão:

- Realizar um comprometimento *root* remoto de um servidor de e-mail.
- Pichar (defacing) um servidor Web.
- Adivinhar e quebrar senhas.
- Copiar um banco de dados contendo números de cartão de crédito.
- Visualizar dados sensíveis (folha de pagamento, registros médicos) sem autorização.

Exemplos de Intrusão (NIST SP 800-61) - Parte 2

- Executar um *packet sniffer* em uma estação de trabalho para capturar nomes de usuário e senhas.
- Usar um erro de permissão em um servidor FTP anônimo para distribuir software pirata e arquivos de música.
- Acessar sistema inseguro e obter acesso à rede interna.
- **Engenharia Social**

O Papel e as Limitações do IDS/IPS

- **IDS: Sistemas de Detecção de Intrusão**
- **IPS: Sistemas de Prevenção de Intrusão**
- **Onde funcionam bem:**
 - São razoavelmente eficazes contra ataques **conhecidos** e **menos sofisticados**.
 - Ex: Ataques de grupos ativistas ou golpes de e-mail em larga escala.
- **Onde falham:**
 - São **menos eficazes** contra ataques sofisticados e direcionados (ex: cibercriminosos ou APTs patrocinados pelo Estado).
 - *Por quê?* Esses atacantes usam **zero-day exploits** e sabem como **obscurecer** suas atividades no sistema.
- **Defesa em Profundidade**
 - O IDS/IPS precisa ser parte de uma estratégia de **Defesa em Profundidade**. Incluindo criptografia, trilhas de auditoria, autenticação forte e gerenciamento ativo de segurança.

Comportamento do Intruso: Metodologia de Ataque

Padrões de comportamento dos intrusos mudam e constantemente

- **Porém existe** uma metodologia de ataque comum:
 - Phishing → Instalação de Malware → Roubo de Credenciais → Comprometimento.

Etapas:

- **1. Aquisição de Alvo e Coleta de Informações**
 - Ossint - Open source intelligence
 - Usa informações publicamente disponíveis (técnicas ou não)
 - Uso de ferramentas de mapeamento de rede
 - **Objetivo:** Identificar e caracterizar os sistemas-alvo.
- **2. Acesso Inicial**
 - Explorando uma vulnerabilidade de rede remota.
 - Explorando credenciais de autenticação fracas.
 - Instalando malware (via **engenharia social** ou **download direcionado**).

- **3. Escalação de Privilégio**

- Ações tomadas **dentro** do sistema **após o acesso inicial**.
- Explora vulnerabilidades de acesso *local*.
- **Objetivo:** Aumentar os privilégios disponíveis para o atacante, permitindo que ele atinja seus objetivos no sistema-alvo.

- **4. Coleta de Informações ou Exploração do Sistema**

- Ações do atacante para acessar ou modificar informações/recursos no sistema.
- Pode incluir mudar para outro sistema-alvo (**movimentação lateral**).

- **5. Manutenção do Acesso**

- Ações para permitir acesso contínuo (persistência) pelo atacante.
- Instalação de **backdoors** ou outro software malicioso.
- Adição de credenciais de autenticação "secretas" .
- Outras mudanças de configuração no sistema.

- **6. Apagando Rastros**

- O atacante desabilita ou edita **logs de auditoria** para remover evidências da atividade de ataque.
- Usa **rootkits** e outras medidas para esconder arquivos ou códigos instalados secretamente.

Exemplos de Comportamento do Intruso

- **(a) Aquisição de Alvo e Coleta de Informações**
 - Explorar site corporativo (estrutura, pessoal, S.O.).
 - Usar DNS lookup (`dig`, `host`) e banco de dados WHOIS.
 - Mapear serviços de rede (NMAP).
 - Enviar e-mail de "sondagem" (ex: SAC) para analisar a resposta (tipo de cliente/servidor de e-mail).
 - Identificar serviços vulneráveis (ex: um CMS Web).
- **(b) Acesso Inicial**
 - *Brute force* (adivinhação) de senha (ex: do CMS).
 - Explorar uma vulnerabilidade em um plugin (ex: do CMS) para ganhar acesso ao sistema.
 - Enviar e-mail de *spear-phishing* direcionado (com link para um *exploit* de navegador).

- **(c) Escalação de Privilégio (Privilege Escalation)**

- Escanear o sistema por aplicações com *exploits* locais.
- Explorar uma aplicação vulnerável para ganhar privilégios elevados (ex: root/admin).
- Instalar *sniffers* para capturar senhas de administrador na rede.
- Usar a senha de admin capturada para acessar informação privilegiada.

- **(d) Coleta de Informações ou Exploração do Sistema**

- Escanear arquivos em busca da informação desejada (ex: dados financeiros, PII).
- Transferir um grande número de documentos para um repositório externo (exfiltração de dados).
- Usar senhas (capturadas) para acessar outros servidores na rede (movimentação lateral).

Exemplos de Comportamento do Intruso

- **(e) Manutenção de Acesso**

- Instalar uma ferramenta de administração remota (RAT) ou *rootkit* com um *backdoor* para acesso futuro.
- Usar a senha de administrador (capturada) para acessar a rede posteriormente (persistência).
- Modificar ou desabilitar programas de antivírus (AV) ou IDS rodando no sistema para evitar detecção.

- **(f) Apagando Rastros**

- Usar um *rootkit* para esconder os arquivos (ex: RAT, sniffers) instalados no sistema.
- Editar arquivos de log (*logfiles*) para remover as entradas (evidências) geradas durante a intrusão.

Detecção de Intrusão (Intrusion Detection): Definições

- **Intrusão de Segurança:**

- Um ato **não autorizado** de **burlar (bypassing)** os mecanismos de segurança de um sistema.

- **Detecção de Intrusão :**

- Uma função de hardware ou software que **coleta e analisa** informações de várias áreas (dentro de um computador ou rede).
- **Objetivo:** Identificar possíveis intrusões de segurança.

Componentes Lógicos de um IDS

- Um IDS (Intrusion Detection System) é composto por três componentes lógicos:
- **1. Sensores**
 - Responsáveis por **coletar dados**.
 - A entrada pode ser qualquer parte do sistema que contenha evidência de intrusão
 - Ex: pacotes de rede, arquivos de log, system calls.
 - Os sensores coletam e encaminham esta informação para o Analisador.
- **2. Analisadores**
 - Recebem a entrada de um ou mais Sensores.
 - A saída **determina se uma intrusão ocorreu**.
 - Pode incluir evidências e orientações sobre ações.
 - Os dados dos sensores também podem ser armazenados para análise futura.
- **3. Interface de Usuário**
 - Permite ao administrador **visualizar** o sistema ou **controlar o seu comportamento**.

Classificação de IDSs (Baseado na Fonte de Dados)

Arquitetura:

- **Simple:** Um IDS com um único sensor e analisador - Ex: um HIDS ou NIDS.
- **Distribuída:** Uso de **múltiplos sensores** em vários hosts e dispositivos de rede, enviando informações para um **analisador centralizado**.

Classificação (Fonte e Tipo de Dados):

- **1. Host-based IDS (HIDS)**
 - **O que monitora?** As características de um **único host** e os eventos ocorrendo nele.
 - *Exemplos:* Identificadores de processo (PIDs), *system calls* (chamadas de sistema).
- **2. Network-based IDS (NIDS)**
 - **O que monitora?** O **tráfego de rede** de segmentos ou dispositivos específicos.
 - *Exemplos:* Analisa tráfego da rede para identificar atividades suspeitas.
- **3. Distributed ou Hybrid IDS (Distribuído/Híbrido)**
 - **O que faz?** Combina informações de vários sensores (HIDS e/ou NIDS) em um analisador central.
 - *Vantagem:* Consegue identificar e responder melhor à atividade de intrusão.

Princípios Básicos: Motivações para o IDS

- Autenticação, Controle de Acesso e Firewalls ajudam a combater intrusões.
- A **Deteccção de Intrusão (IDS)** é outra linha de defesa, motivada por várias considerações:
 - **1. Deteccção Rápida**
 - Se a intrusão for detectada rápido o suficiente, o intruso pode ser **ejetado** do sistema *antes* de causar danos ou comprometer dados.
 - Mesmo se não for a tempo de "preempt" (impedir), quanto mais cedo a detecção, menor o dano e mais rápida a recuperação.
 - **2. Efeito desencorajador**
 - Um IDS eficaz pode desencorajar ataques, atuando assim para **prevenir** intrusões.
 - **3. Coleta de Informações**
 - A detecção permite a coleta de informações sobre **técnicas de intrusão**.
 - Essas informações podem ser usadas para fortalecer as medidas de **prevenção** (ex: regras de firewall, patches).

Princípios Básicos: A Suposição Fundamental do IDS

- **A Suposição:**

- A detecção de intrusão baseia-se na suposição de que o comportamento do **intruso** difere do comportamento de um **usuário legítimo** de maneiras que podem ser **quantificadas**.

- **O Desafio (A Realidade):**

- Não podemos esperar que haja uma distinção **nítida (crisp)** e **exata** entre um ataque e o uso normal dos recursos.

- **A Consequência (Falsos Positivos/Negativos):**

- Devemos esperar que haverá alguma **sobreposição (overlap)** entre o comportamento legítimo e o comportamento de ataque.
- (O trabalho do Analisador é minimizar essa sobreposição).

O Desafio Central do IDS: O "Trade-Off"

- O comportamento típico de um intruso e de um usuário autorizado não são perfeitamente distintos; existe uma **sobreposição (overlap)**.
- **O Dilema (Compromisso):**
 - **1. Interpretação "Frouxa" (Loose):**
 - Tenta capturar o máximo de intrusos.
 - **Resultado:** Leva a um alto número de **Falsos Positivos** (Falsos Alarmes), onde usuários autorizados são identificados como intrusos.
 - **2. Interpretação "Restrita" (Tight):**
 - Tenta limitar os Falsos Positivos.
 - **Resultado:** Leva a um aumento de **Falsos Negativos** (intrusos não identificados).
- **O Objetivo Ideal:**
 - A prática do IDS é um "compromisso" (e uma "arte").
 - Maximizar a **Taxa de Detecção** (detectar ataques reais).
 - Minimizar a **Taxa de Falsos Alarmes** (ignorar uso normal).

A dificuldade de detecção depende do tipo de intruso:

- **Detectando Atacantes Externos (Outsiders):**

- É possível (com confiança razoável) distinguir um atacante externo de um usuário legítimo.
- **Como?** Padrões de comportamento legítimo podem ser estabelecidos (observando o histórico).
- **Detecção:** Pode-se detectar **desvios significativos** (anomalias) desses padrões.

- **Detectando Atacantes Internos (Insiders):**

- (Um usuário legítimo agindo de forma não autorizada).
- Anderson concluiu que esta é a tarefa **mais difícil**.
- **Por quê?** A distinção entre o comportamento "anormal" (malicioso) e "normal" (legítimo) de um insider pode ser **muito pequena**.
- **Conclusão:** A busca *apenas* por anomalias seria **insuficiente** para detectar insiders.
- **Solução (sugerida):** O comportamento do insider pode ser detectável através de uma **definição inteligente** de condições (regras) que sugerem uso não autorizado.

- **O Objetivo Duplo (e Conflitante):**
 - **1. Alta Taxa de Detecção:** Detectar uma porcentagem substancial de intrusões (senão, o IDS gera uma *falsa sensação de segurança*).
 - **2. Baixa Taxa de Falsos Alarmes:** Se os falsos positivos (alarmes) forem frequentes, os gerentes **passarão a ignorar os alarmes** ou perderão muito tempo analisando-os.
- **A Falácia da Taxa-Base (Base-Rate Fallacy):**
 - É **extremamente difícil** atingir os dois objetivos.
 - **O Problema:** O número real de **intrusões** (taxa-base) é **muito baixo** em comparação com o número de **usos legítimos** (normal).
 - **A Consequência:** A menos que o teste (o IDS) seja *extremamente discriminatório* (quase perfeito), a taxa de falsos alarmes será **alta**.
 - **O desafio está aqui**

Requisitos de um IDS - Parte 1

Um IDS deve:

- **1. Rodar continuamente** com mínima supervisão humana.
- **2. Ser tolerante a falhas**, ou seja, capaz de se recuperar de *crashes* e reinicializações.
- **3. Resistir à subversão.** O IDS deve ser capaz de monitorar a si mesmo e detectar se foi modificado por um atacante.
- **4. Impor um overhead mínimo** no sistema onde está rodando.
- **5. Ser configurável** de acordo com as políticas de segurança do sistema que está sendo monitorado.

sobrecarga

- **6. Ser capaz de se adaptar** a mudanças no comportamento do sistema e dos usuários ao longo do tempo.
- **7. Ser capaz de escalar** para monitorar um grande número de hosts.
- **8. Evitar parada completa** do serviço, ou seja, se alguns componentes pararem, o resto deve ser afetado o mínimo possível.
- **9. Permitir reconfiguração dinâmica**, ou seja, a capacidade de reconfigurar o IDS sem ter que reiniciá-lo.

Os IDSs usam duas abordagens:

- **1. Detecção de Anomalia**
- **2. Detecção por Assinatura ou Heurística**

- 1. Detecção de Anomalia

- **Como funciona?** Envolve a coleta de dados sobre o comportamento de **usuários legítimos** ao longo do tempo (criando uma "linha de base" do que é **normal**).
- **Análise:** O comportamento *atual* observado é então analisado.
- **Decisão:** O IDS determina se o comportamento atual é de um usuário legítimo ou se é uma **anomalia** desse padrão, indicando um intruso.

Abordagem de Análise: Detecção de Anomalia (Definição)

Duas fases:

- **1. Fase de Treinamento**

- Desenvolve um **modelo** do comportamento legítimo (normal) do usuário.
- Como? Coletando e processando dados dos sensores durante a operação normal do sistema.
- (Pode ocorrer em momentos distintos ou ser um processo contínuo de evolução do modelo).

- **2. Fase de Detecção**

- O comportamento **atual** observado é comparado com o modelo (normal).
- O comportamento é então classificado como **legítimo** ou **anômalo** (suspeito).

Detecção de Anomalia: Categorias de Classificação

Temos três abordagens:

- **1. Estatística**
 - Análise do comportamento observado usando modelos (univariados, multivariados, time-series) das métricas.
- **2. Baseada em Conhecimento**
 - Usa um **sistema especialista** (expert system) que classifica o comportamento de acordo com um conjunto de **regras** que modelam o comportamento legítimo.
- **3. Aprendizado de Máquina (Machine-learning)**
 - Determina **automaticamente** um modelo de classificação adequado a partir dos dados de treinamento, usando técnicas de mineração de dados.

Abordagem 1: Estatística

Desenvolve um **perfil estatístico** das métricas observadas.

- **Uso de:**

- *Univariada*: Trata cada métrica como independente (muito "cru" (crude), ineficaz).
- *Multivariada*: Considera correlações entre métricas (melhor discriminação).
- *Séries Temporais (Time-series)*: Usa a ordem e o tempo entre eventos (ainda melhor).

- **Vantagens:**

- Relativa simplicidade e baixo custo computacional.
- Não necessita de suposições (assumptions) sobre o comportamento esperado.

- **Desvantagens:**

- Dificuldade em selecionar métricas adequadas (balanço de falsos positivos/negativos).
- Nem todos os comportamentos podem ser modelados por esta abordagem.

Abordagem 2: Baseada em Conhecimento

- Classifica os dados observados usando um **conjunto de regras** (expert system).
- **Fase de Treinamento:**
 - As regras são desenvolvidas (podendo ser **manualmente**) para caracterizar os dados de treinamento em classes distintas.
 - Pode usar ferramentas formais: máquinas de estado finito, linguagens de descrição.
- **Vantagens:**
 - Robustez e flexibilidade.
- **Desvantagens:**
 - **Dificuldade e tempo** para desenvolver regras (conhecimento) de alta qualidade.
 - Necessidade de **especialistas humanos** (human experts) para auxiliar no processo.

Abordagem 3: Aprendizado de Máquina (Conceito e Trade-offs)

- Usa técnicas de *data mining* para desenvolver **automaticamente** um modelo a partir dos dados de treinamento (normais).
- **Fase de Treinamento (Custo):**
 - Requer **tempo significativo** e **altos recursos computacionais**.
- **Fase de Detecção (Eficiência):**
 - Uma vez que o modelo é gerado, a análise subsequente (classificação) é geralmente **bastante eficiente**.
- **Vantagens:**
 - Flexibilidade e adaptabilidade.
 - Capacidade de capturar interdependências complexas entre as métricas.
- **Desvantagens:**
 - Dependência de suposições sobre o comportamento "aceito".
 - Taxa de **falsos alarmes** (false alarm rate) **inaceitavelmente alta** (atualmente).
 - Alto custo de recursos (para treinar).

Abordagem 3: Aprendizado de Máquina - Abordagens

Várias abordagens de ML foram tentadas com sucesso variado:

- **Redes Bayesianas:** Codifica relações probabilísticas.
- **Modelos de Markov:** Modelo com conjuntos de estados interligados por probabilidades de transição.
- **Redes Neurais:** Para classificar dados.
- **Lógica Fuzzy:** Raciocínio é aproximado; pode acomodar incerteza.
- **Algoritmos Genéticos:** Desenvolver regras de classificação.
- **Clusterização e Detecção de Outlier:** Agrupa dados por similaridade; novos dados são classificados como "dentro de um cluster" (normal) ou "outlier" (anomalia).

- **2. Detecção por Assinatura ou Heurística**

- Também conhecida como **Detecção de Mau Uso** .
- **Como funciona?** Usa um conjunto de **padrões maliciosos conhecidos** ou **regras de ataque**.
- **Análise:** O comportamento atual é comparado com este conjunto de regras/assinaturas.
- **Decisão:** Se houver uma correspondência, é considerado um intruso.

Abordagem de Análise: Detecção por Assinatura

- **Como Funciona:**

- Compara os dados coletados com **padrões conhecidos de dados maliciosos**.
- As assinaturas precisam dar detalhes o suficiente para minimizar falsos alarmes, mas ainda detectar uma fração suficiente de dados maliciosos.

- **Onde é usado:** Amplamente utilizado em produtos antivírus, proxies de varredura de tráfego de rede e NIDS.

- **Vantagens:**

- Custo relativamente baixo em tempo e uso de recursos.
- Ampla aceitação.

- **Desvantagens:**

- Esforço significativo necessário para identificar e revisar novos comportamentos para criar assinaturas.
- **Incapacidade de detectar ataques "zero-day"** onde nenhuma assinatura existe.

Abordagem de Análise: Detecção Heurística

- **Como Funciona:**

- Envolve o uso de **regras** para identificar ataques **conhecidos** ou que explorariam **fraquezas conhecidas**.
- As regras também podem identificar comportamento **suspeito**, mesmo que esse comportamento esteja dentro dos padrões de uso estabelecidos.

- **Desenvolvimento das Regras:**

- Abordagem mais frutífera: **Analisar ferramentas e scripts de ataque** coletados na Internet.
- Suplementado por regras geradas por **especialistas**.
- As regras são tipicamente específicas para a máquina e o S.O.

- **Exemplo:**

- O sistema **SNORT** é um exemplo de um **NIDS baseado em regras**.
- Existe uma grande coleção de regras para ele detectar uma ampla variedade de ataques de rede.

Host-Based IDS (HIDS): Introdução

- **O que é?**
 - O HIDS adiciona uma **camada especializada de software de segurança** a sistemas vulneráveis ou sensíveis.
 - *Exemplos:* Servidores de banco de dados, sistemas administrativos.
- **Função:**
 - Monitora a atividade **dentro do sistema** de várias maneiras para detectar comportamento suspeito.
- **Propósito Principal:**
 - 1. Detectar intrusões.
 - 2. Registrar (log) eventos suspeitos.
 - 3. Enviar alertas.
 - IDS/IPS
- **Abordagens de Análise:**
 - HIDS usa as duas abordagens:
 - Detecção de Anomalia (Anomaly).
 - Detecção por Assinatura ou Heurística.

HIDS: Fontes de Dados e Sensores (Parte 1)

- Um componente fundamental do IDS é o **sensor** que coleta dados. Um registro da atividade (input) deve ser fornecido ao analisador.
- Fontes de dados comuns incluem:
- **1. Rastros de Chamadas de Sistema**
 - Um registro da sequência de *system calls* feitas pelos processos.
 - Amplamente reconhecida como a **fonte de dados preferida** para HIDS.
 - **Vantagem:** Funciona bem em sistemas Unix e Linux.
 - **Desvantagem:** Problemático em sistemas Windows (devido ao uso extensivo de DLLs, que "obscurecem" quais processos usam quais chamadas).
- **2. Registros de Auditoria**
 - A maioria dos S.O. modernos inclui software de "accounting" (auditoria) que coleta informações sobre a atividade do usuário.
 - **Vantagem:** Nenhum software de coleta adicional é necessário.
 - **Desvantagem (1):** Os registros podem não conter a informação necessária ou não estar em um formato conveniente.
 - **Desvantagem (2):** Intrusos podem tentar **manipular** esses logs para esconder suas ações.

- **3. Checksums de Integridade de Arquivos**

- Abordagem comum: Escanear periodicamente **arquivos críticos** (ex: arquivos de sistema) em busca de mudanças.
- **Como?** Comparando hash atuais com um registro de valores conhecidos (baseline).
- **Desvantagem (1):** A necessidade de gerar e proteger os checksums "bons" (baseline).
- **Desvantagem (2):** A dificuldade de monitorar arquivos que *mudam* (legitimamente) o tempo todo.

- **4. Acesso ao Registro**

- Uma abordagem usada em sistemas **Windows**.
- Monitora o acesso ao Registro (dada a quantidade de informação e acesso usados pelos programas).
- **Desvantagem:** Muito específico do Windows; registrou sucesso limitado.

- **Função Resumida do Sensor:**

- Coleta dados → Filtra (padroniza formato) → Encaminha para o Analisador.

HIDS por Anomalia (Sistemas Linux/UNIX)

- A maioria dos trabalhos em HIDS por anomalia foi feita em UNIX/Linux (dada a facilidade de coletar dados).
- **Fonte de Dados Preferida: Rastros de Chamadas de Sistema .**
 - Logs de auditoria.
- **Por que "System Calls" ?**
 - São o meio pelo qual os programas acessam as funções do **kernel**.
 - Fornecem informação **detalhada** sobre a atividade do processo, permitindo classificar como normal ou anômalo.

- Os *system call traces* (coletados) são analisados por um motor de decisão.
- **Abordagem Original (STIDE):**
 - Compara sequências de chamadas observadas com sequências "normais" (do treinamento) para obter uma "taxa de disparidade" (mismatch ratio).
- **Abordagens Alternativas (ML):**
 - Hidden Markov Models (HMM)
 - Redes Neurais Artificiais (ANN)
 - Support Vector Machines (SVM)
 - Extreme Learning Machines (ELM)
- **Performance:**
 - taxa% detecção, falsos positivos.
 - velocidade de detecção.

HIDS: Detecção por Assinatura ou Heurística

- **Implementação Comum:** É a base dos **Antivírus**.
 - Usado em: sistemas cliente (PCs), dispositivos móveis, e incorporado em proxies (e-mail, Web) e NIDS.
- **Como Funciona (Duas Técnicas):**
 - **1. Assinaturas :**
 - Um banco de dados de **padrões de dados** (file signatures) encontrados em malware conhecido.
 - **2. Heurísticas:**
 - Um conjunto de **regras** que caracterizam o **comportamento** malicioso conhecido.
- **Vantagem:**
 - *Muito eficiente* para detectar malware conhecido.
- **Desvantagem :**
 - **NÃO é capaz** de detectar ataques "**zero-day**" (ataques novos/desconhecidos).
 - Pois o novo ataque não corresponde a nenhuma assinatura ou regra conhecida.
- **Uso Principal:**
 - Amplamente utilizado, especialmente em sistemas **Windows** (que continuam a ser um alvo principal de intrusos).

- **O Foco Tradicional:**

- O trabalho em HIDS (Host-based) focava-se na operação **stand-alone** (em um único sistema).

- **A Realidade Organizacional:**

- A organização típica precisa defender uma coleção **distribuída** de hosts (suportados por uma LAN ou inter-rede).

- **A Evolução (A Defesa Eficaz):**

- Embora seja possível usar HIDS *stand-alone* em cada host...
- Uma defesa **mais eficaz** é alcançada através da **coordenação e cooperação** entre os IDSs pela rede (ou seja, um HIDS Distribuído).

Network-Based IDS (NIDS): Definição e Papel

- **O que é um NIDS?**

- Monitora o tráfego em **pontos selecionados** de uma rede.
- Examina o tráfego **pacote a pacote** em tempo real (ou próximo a isso).
- Analisa atividade nos protocolos de Rede (L3), Transporte (L4) e/ou Aplicação (L7).

- **Contraste (NIDS vs. HIDS):**

- **NIDS:** Examina o *tráfego de pacotes* **direcionado aos** sistemas.
- **HIDS:** Examina a *atividade do usuário e do software* **dentro de** um host.

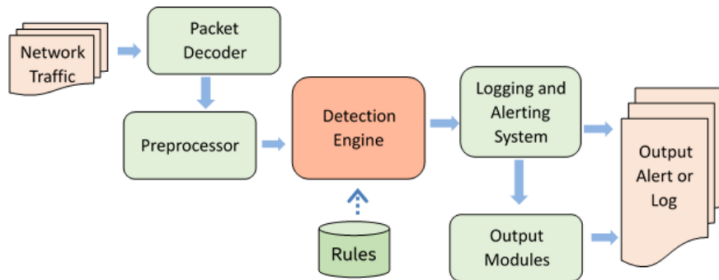
- **Papel e Localização:**

- Tipicamente incluído na infraestrutura de **segurança de perímetro** (incorporado ou associado ao firewall).
- Foco: Monitorar tentativas de intrusão **externa**.

- **Arquitetura Típica:**

- Sensores (para monitorar tráfego).
- Servidores de Gerenciamento (para análise).
- Consoles de Gerenciamento (para interface humana).

Funcionamento NIDS



NIDS: A Limitação da Criptografia e Modos de Sensor

- **A Grande Limitação - Criptografia:**

- Com o uso crescente de criptografia (ex: TLS/SSL), os NIDS **perderam o acesso ao conteúdo (payload)** significativo do tráfego.
- Isso **prejudica (hindering)** sua capacidade de funcionar bem (ex: não podem ver comandos maliciosos dentro de uma sessão HTTPS).
- **Conclusão:** NIDS são importantes, mas só podem formar **parte** da solução (Defesa em Profundidade).

- **Tipos de Sensores de Rede (Modos):**

- **1. Sensor Inline**

- Inserido em um segmento de rede; o tráfego **deve passar através** do sensor.
- (Pode ser combinado com um firewall ou switch).
- **Vantagem (IPS):** Permite **bloquear** um ataque quando detectado (atuando como IDS e IPS - Intrusion Prevention System).

- **2. Sensor Passivo**

- (Mais comum).
- Monitora uma **cópia** do tráfego de rede; o tráfego real *não* passa pelo dispositivo.
- **Vantagem:** Mais eficiente, pois não adiciona um passo de manuseio extra (não contribui para o atraso/delay dos pacotes).

NIDS: Configuração Passiva e Sensores Wireless (WIDS)

- **Configuração Típica**

- O sensor se conecta ao meio (ex: cabo de fibra óptica) através de um **"tap"** (derivação física).
- O "tap" fornece ao sensor uma **cópia** de todo o tráfego da rede.
- **Uso de Placas de Rede:**
 - **NIC 1 (Coleta):** Conectada ao "tap". Geralmente **não possui um endereço IP**. Apenas coleta todo o tráfego (modo promíscuo).
 - **NIC 2 (Gerenciamento):** Conectada à rede **com um endereço IP**. Permite ao sensor se comunicar com o servidor de gerenciamento NIDS.

- **Distinção: Sensores Wireless**

- Pode ser *inline* (incorporado em um Access Point - AP) ou *passivo* (monitorando o tráfego "pelo ar").
- **Função:** Apenas sensores wireless podem analisar o tráfego de **protocolo wireless** e detectar ataques específicos (ex: DoS wireless, sequestro de sessão, AP falso).
- **WIDS (Wireless IDS):** Um NIDS focado exclusivamente em redes sem fio.

Honeypots: Definição e Propósito

- Honeypots (Potes de Mel) são **sistemas-isca (decoy systems)**.
- São projetados para **atrair (lure)** um atacante potencial para **longe** dos sistemas críticos.
- **Três Objetivos Principais:**
 - **1. Desviar (Divert):** Desviar um atacante de acessar sistemas críticos.
 - **2. Coletar Informações:** Coletar dados sobre a atividade do atacante (técnicas, ferramentas).
 - **3. Ganhar Tempo (Encourage):** Encorajar o atacante a ficar no sistema tempo suficiente para que os administradores possam responder.
- **A Lógica Fundamental:**
 - O honeypot é preenchido com **informações fabricadas** (que parecem valiosas, mas um usuário legítimo nunca acessaria).
 - **Qualquer acesso** ao honeypot é, por definição, **suspeito**.
 - O sistema é instrumentado com monitores e loggers sensíveis.
 - Como o ataque "parece" ser bem-sucedido, os admins podem rastrear o atacante sem expor sistemas produtivos.

Honeypots: Tipos de Interação (Low vs. High)

- **Valor do Honeypot:** É um recurso sem valor de produção. Não há razão legítima para interação.
 - Qualquer comunicação *de entrada* (inbound) é (provavelmente) uma sonda, scan ou ataque.
 - Se um honeypot inicia comunicação *de saída* (outbound), ele (provavelmente) foi comprometido.
- **Classificação (Nível de Interação):**
- **1. Low Interaction Honeypot (Baixa Interação)**
 - Um pacote de software que **EMULA** serviços ou sistemas (ex: finge ser um servidor FTP).
 - Fornece uma interação inicial realista, mas **não executa** a versão completa dos serviços.
- **2. High Interaction Honeypot (Alta Interação)**
 - É um **sistema real**: S.O. completo, serviços e aplicações reais.
 - O sistema é instrumentado (monitorado) e colocado onde os atacantes podem acessá-lo.

Trade-offs (High vs. Low) e Honeynets

- **High Interaction (Vantagem):**
 - Um alvo **muito mais realista**.
 - Pode "ocupar" (prender) um atacante por um período extenso.
- **High Interaction (Desvantagens / Riscos):**
 - Requer **significativamente mais recursos** (é um S.O. real).
 - **RISCO:** Se for comprometido, pode ser usado pelo atacante para iniciar ataques contra **outros sistemas** (na Internet).
 - Isso pode resultar em problemas **legais** ou de **reputação** indesejados para a organização.
- **Low Interaction (Vantagem):**
 - Suficiente para detectar estágios iniciais do ataque (scan, probe) e alertar (como componente de um IDS distribuído).
- **Honeynet (Rede Honeypot):**
 - Esforços recentes ("The Honeynet Project") focam em construir **redes inteiras** de honeypots que emulam uma empresa (com tráfego simulado).

Implantação (Deployment) de Honeypots

- A localização depende dos objetivos (tipo de info a coletar) e do nível de risco tolerado.
- **1. Externo** (Internet, antes do firewall)
- **2. DMZ** (Rede de Serviços, entre firewalls)
- **3. Interno** (Rede Corporativa, após o firewall)

Implantação: Posição 1 (Externa)

- **Localização:** Na Internet, **antes** do firewall externo.
- **Vantagens:**
 - Útil para rastrear tentativas de conexão a **IPs não utilizados** (scans).
 - **Não aumenta o risco** para a rede interna (o perigo de um sistema comprometido *atrás* do firewall é evitado).
 - **Reduz "ruído":** Atrai muitos ataques, reduzindo os alertas no firewall e nos sensores IDS internos (facilitando o gerenciamento).
- **Desvantagem:**
 - Pouca ou nenhuma capacidade de capturar **atacantes internos**.

Implantação: Posição 2 (DMZ)

- **Localização:** Na rede de serviços (Web, Mail), ou seja, **dentro da DMZ** (entre o firewall externo e o interno).
- **Vantagem (Implícita):**
 - Monitora ataques direcionados aos serviços públicos (Web, Mail, DNS).
- **Desvantagens (Trade-off):**
 - **Risco de Contaminação:** O admin deve garantir que os *outros sistemas* (legítimos) na DMZ estejam seguros contra o honeypot (caso ele seja comprometido).
 - **Conflito com Firewall:** O firewall (externo) tipicamente **bloqueia** tráfego desnecessário para a DMZ (ex: permite só porta 80, 443).
 - **Dilema:** O admin deve (1) **abrir o firewall** (aumentando o risco) ou (2) **limitar a eficácia** do honeypot (que não verá os ataques bloqueados).

Implantação: Posição 3 (Interna) e "Honeyfiles"

- **Localização:** Na rede **totalmente interna**.
 - **Vantagens:**
 - Vantagem mais importante: Pode capturar **ataques internos**.
 - Pode detectar um firewall **mal configurado** (que encaminha tráfego indevido da Internet para a rede interna).
 - **Desvantagens (Sérias):**
 - **Risco Alto:** Se o honeypot for comprometido, ele pode atacar **outros sistemas internos**.
 - (O firewall não bloquearia o tráfego do atacante para o honeypot, pois ele é visto como tráfego "permitido" para o honeypot).
 - (Similar à DMZ, o firewall interno precisa de regras de exceção).
-
- **Tecnologia Relacionada: Honeyfiles**
 - Emulam **documentos** legítimos (com nomes realistas, ex: "Salarios_Diretoria.xlsx").
 - Atuam como "isca" (bait) para intrusos.
 - **Qualquer acesso** a esses arquivos (que não deveriam ser acessados por usuários legítimos) é considerado **suspeito**.

Regra Snort:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET ![7680,1521]
(msg:"ET P2P BitTorrent peer sync";
 flow:established,to_server;
 content:"|00 00 00 0d 06 00|"; depth:6;
 threshold: type limit, track by_dst, seconds 300, count 1;
 reference:url,bitconjurer.org/BitTorrent/protocol.html;
 classtype:policy-violation; sid:2000334; rev:14;
 metadata:created_at 2010_07_30, confidence Medium,
 signature_severity Informational, updated_at 2025_06_30;)
```



```
alert tcp $HOME_NET any -> $EXTERNAL_NET ![7680,1521]
```

- **alert**: gera um alerta quando a condição é atendida.
- **tcp**: apenas tráfego TCP.
- **\$HOME_NET any**: origem na rede interna, qualquer porta.
- **\$EXTERNAL_NET ![7680,1521]**: destino na rede externa, exceto portas 7680 e 1521.

Opções da Regra (1/2)

```
msg:"ET P2P BitTorrent peer sync";  
flow:established,to_server;  
content:"|00 00 00 0d 06 00|"; depth:6;
```

- **msg:** Mensagem exibida nos logs.
- **flow:established,to_server:** Só considera conexões TCP já estabelecidas, indo para o servidor.
- **content:** Procura a sequência de bytes 00 00 00 0d 06 00.
- **depth:6:** Pesquisa apenas nos primeiros 6 bytes do payload.

Opções da Regra (2/2)

```
threshold:  type limit, track by_dst, seconds 300, count 1;  
reference:url,bitconjurer.org/BitTorrent/protocol.html;  
classtype:policy-violation; sid:2000334; rev:14;
```

- **threshold:** Evita múltiplos alertas — 1 alerta por destino a cada 300s.
- **reference:** Link para a documentação do protocolo BitTorrent.
- **classtype:** Categoria: violação de política.
- **sid:** Identificador único (2000334).
- **rev:** Revisão nº 14.